



Enhanced code reviews using pull request based change impact analysis

Ismail Seren Göçmen¹ · Ahmed Salih Cezayir¹ · Eray Tüzün¹

Accepted: 3 December 2024 / Published online: 12 February 2025
© The Author(s) 2025

Abstract

Context Code reviewing is an essential yet challenging activity due to the potential repercussions associated with changes to the codebase. While version control highlights differences between versions, it may not adequately alert developers to potential side effects of changes. Change impact analysis alongside code differences can guide reviewers in making informed decisions.

Objective We aim to enhance the code review process by providing helpful insights to reviewers by analyzing change impact.

Method We propose a novel change impact analysis approach with the granularity of pull requests by combining call graph-based dependency analysis and history mining techniques. By utilizing these, we calculate several file metrics and an overall risk score for each pull request. To validate the approach, we conducted two focus group sessions, including a feature feedback survey, a tool demo, a post-demo survey, and focus group discussions. Additionally, we performed experiments on three open-source projects to evaluate the computational feasibility.

Results From the focus groups, our approach received an average of 3.66 out of 5.0 for enhancing the code review experience across five aspects. The current risk score formula averaged 3.2 out of 5.0 for accurately representing change impact, with 71% agreement on the selected metrics. Experiments confirmed computational viability, with analysis times ranging from 7.4 to 22.43 seconds.

Conclusions Our study underscores the potential benefits of integrating change impact analysis into the code review process. By leveraging this approach, developers can conduct more

Communicated by: Steffen Herbold, Eunjong Choi

Ismail Seren Göçmen and Ahmed Salih Cezayir contributed equally to the study.

This article belongs to the Topical Collection: *Predictive Models and Data Analytics in Software Engineering (PROMISE)*

✉ Ismail Seren Göçmen
isgocmen@alumni.bilkent.edu.tr

✉ Eray Tüzün
eraytuzun@cs.bilkent.edu.tr

Ahmed Salih Cezayir
acezayir@alumni.bilkent.edu.tr

¹ Bilkent University, Ankara, Turkey

thorough reviews, enhancing their ability to detect and mitigate potential issues arising from code changes.

Keywords Change impact analysis · Code review · Call graphs · Static analysis · Pull request · Risk management

1 Introduction

Changes are prevalent in software projects, and version control systems (VCSs) are used to manage them (Zolkifli et al. 2018). A VCS can help practitioners to protect source code from unintentional human-based errors and their potential impacts with the pull request (PR) mechanism. Code reviewers can look at the changes in the PR and see the obvious errors. However, some errors can be difficult or impossible to identify since VCSs do not show the potential impact of proposed changes (Blischak et al. 2016). **Change impact analysis (CIA) is a way to estimate the possible effects of proposed changes, which can also be utilized to detect hidden errors, thereby significantly reducing the required resources to maintain the project (Sun et al. 2010). Impact analysis is crucial, especially in mission-critical systems, since a single unexpected error can lead to severe consequences (Fowler 2004).**

A variety of CIA approaches have been developed over many years that leverage static analysis (Acharya and Robinson 2011; Buckner et al. 2005; Cuoq et al. 2012; Dai et al. 2022; Hattori et al. 2008), dynamic analysis (Apiwattanapong et al. 2005; Cai and Santelices 2014; Cai and Thain 2016; Wang and Roychoudhury 2004), dependency analysis (Alimadadi et al. 2015; Follett and Hoerber 2010; Rungta et al. 2012; Zhang et al. 2012) and mining software repositories (MSR) (Dit et al. 2014; Hattori et al. 2008; Zimmermann et al. 2005). **In static and dynamic analysis techniques, code pieces, their behaviors, and call relationships are used to measure the impact, whereas, in MSR, fetched historical data from code repositories are utilized to uncover the hidden relationships between different code entities. Yet, to the best of our knowledge, no CIA approaches combine MSR and call graph based dependency analysis at the granularity level of PR to enhance the code review experience by providing additional insights directly within the PR. Performing PR level CIA might help with more accurate impact calculations since related changes are generally combined under a PR in software projects. This approach can be integrated with VCSs to enhance the code review experience.**

Even though there are not many studies about improving code reviews by implementing CIA, there are studies suggesting different methods. Static code analysis is one of the most prominent methods that code review tools use, where source code is examined without executing the code, allowing for the identification of potential issues and vulnerabilities early in the development cycle. Through static code analysis, tools like SonarQube¹ can automatically detect violations of coding standards, flagging deviations from established conventions and best practices. They can also analyze code for performance bottlenecks, security vulnerabilities, and potential code smells, alerting developers to areas that require further attention or optimization. Although static analysis methods are useful, they are not enough to detect the potential impact of proposed changes, alone.

This paper reports on a novel CIA approach and its tool implementation, Change Impact Detector (CHID). The primary purpose of our approach is to enhance the code review experience by utilizing static analysis methods with predetermined metrics. We utilize a project-wide function call graph to reveal call dependencies to detect potentially impacted

¹ <https://www.sonarsource.com/products/sonarqube>

functions. Then, we fetch information about each PR from the software repository to analyze code churn, bug frequency, co-changed files, author merge rate, PR size (lines of code), and impact size (in terms of page rank score). A risk score is calculated as a weighted sum of previous metrics. Lastly, the analysis results are presented as a detailed comment directly within the corresponding PR on GitHub, providing reviewers with immediate, contextual insights.

The goal of this paper is further divided into two research questions.

– **RQ1. How effective is analyzing change impact on the PR level for enhancing the code review experience?**

Our approach can be applied to any goal-oriented set of changes, such as a commit, a PR, or a version release, with minor variations in metric calculations. For this study, we focus on the PR level, as it is one of the most widely used units in code review practices (Yu et al. 2016). By PR level, we refer not only to the pull request itself but also to its associated files and the methods modified within its changeset. This allows us to analyze the impact at a detailed level that matches how developers typically manage and review changes. The goal of this research question is to assess the effects of CIA on the code review experience, specifically from the developers' perspective.

– **RQ2. How effective is the introduced risk score approach for indicating the change impact of a PR?**

– **RQ2.1 Does the introduced risk formula correctly represent the change impact of a PR?**

– **RQ2.2 Does the representation of the risk score thoroughly reflect the change impact of a PR?**

This question is divided into two sub-questions. The first sub-question concentrates on whether the introduced risk score approach successfully assesses the risk level of a given PR. The second sub-question focuses on whether the potential impact can be accurately represented by the current risk score visual representation.

To validate the applicability of our approach, we conducted two focus group sessions with software practitioners to analyze the applicability and effectiveness of the proposed change impact calculation method for enhancing the code review experience. In addition to assessing user feedback, we performed experiments on three open-source projects to evaluate the computational feasibility of our approach. Since the primary goal of our research is to assist developers during development, CHID has been integrated closely with GitHub, a widely used version control platform.

The main contributions of this paper are as follows:

1. Demonstration of the utility of the newly introduced CIA approach on the PR level, which combines MSR with call graph based dependency analysis
2. Introduction of a heuristic risk score calculation mechanism that represents the potential of a PR causing side effects that may lead to unexpected behavior throughout program execution
3. Development of a tool, CHID, which uses the depicted CIA approach to enhance the code review experience of projects that use Java 8 and GitHub as for version control

The remainder of this paper is organized as follows. In the following section, we present the background information. In Section 3, we introduce our approach. Section 4 presents the implementation details of the proposed CIA technique. We describe the focus group study design in Section 5. Section 6 presents the results of focus group sessions and the

computational feasibility experiments. Section 7 discusses the insights from the obtained results. Section 8 includes limitations and threats to the validity of the study. Section 9 presents related work, and finally, Section 10 concludes our study.

2 Background Information

To reflect the two primary areas our proposed approach focuses on CIA and code review experience—we have organized this section into two subsections. While we discuss CIA and code review related studies here, the main comparison with tools that specifically leverage CIA for code reviews will be addressed in Section 9.

2.1 Studies on Change Impact Analysis

CIA, formerly called ripple effect, is a technique to determine possible effects resulting from code modifications (Bohner and Arnold 1996). Haney's paper from 1972, which focuses on module connection analysis, is known as the first paper related to impact analysis (Haney 1972). The introduced technique states that every change in one of the modules in a module pair will lead to the necessity of a change in the other. Program slicing, which is also another CIA technique, was introduced by Weiser (1979). It is used to determine dependencies between different code pieces. Later, this technique is used to trace requirements since it can also find dependencies among different documents. The term "ripple effect", first introduced by Yau et al. (1980), refers to the potential consequences of a modification and is used to evaluate a program's stability in response to changes. In 1996, a series of research articles were published by Arnold and Bohner, which present the current state of CIA techniques at that time (Bohner and Arnold 1996). Most studies published between 1996 and 2005 focused on adapting existing strategies and techniques to new concepts (Jönsson et al. 2005). Below, we describe studies about the CIA utilizing different techniques which were published after 2005.

Acharya and Robinson (2011) presented a framework implementation, Imp, which decreases the required time to assess the change impact with the help of a static program slicing technique. By evaluating their framework on a large codebase, they concluded that their framework carefully addresses time and accuracy tradeoffs by selecting appropriate settings for the selected change cluster.

Dai et al. (2022) introduced an analysis technique by merging program slicing and call graphs to investigate the multi-granularity change impact of C programs. They created a supporting tool, CPCIA, and analyzed five programs to assess their technique. Their findings showed that their technique has good precision and can accurately analyze the impact of both method and statement level changes.

Cai and Santelices (2014) proposed a dynamic impact analysis technique called DIVER that traces the change impact of methods by dependency analysis. They compared their technique with an optimized PathImpact (Law and Rothermel 2003) implementation that leverages execute-after sequences in four Java programs. They concluded that Diver prevents 70% of the false positives included in the result set of PathImpact analysis.

Cai and Thain (2016) presented DistIA, a dynamic analysis technique that forecasts proliferated impacts inside and outside process boundaries by partly sorting distributed method-execution events. By analyzing six Java programs and comparing the results with real impact sets, they determined DistIA had 71.2% precision and 100% recall for any query.

Alimadadi et al. (2015) implemented Tochal, a tool that performs a DOM-sensitive CIA for JavaScript by merging static and dynamic call graphs. Their technique ranks the importance of the entities in the detected impact set based on a dependency graph created by Tochal. They conducted a controlled experiment with 10 participants utilizing predetermined tasks. The experimental group using Tochal completed tasks 78% faster and 223% more accurately than the control group.

Ren et al. (2004) proposed Chianti, which detects interdependent small changes from different program versions to build call graphs for test suites. Then, it determines potentially impacted methods and relevant affecting changes using affected tests.

Buckner et al. (2005) developed an Eclipse plug-in, JRipples, that uses static information to analyze dependencies between entities to help developers locate the impact set by keeping track of visited elements and the elements that are dependent on them.

Zhang et al. (2012) introduced FaultTracer, which is a toolkit that determines atomic changes from abstract syntax trees (AST) of two program versions, finds their dependencies by tracking reference of each called method, runs selected tests to emphasize failure-inducing changes and ranks these changes by using spectrum-based fault localization technique for Java programs.

Zimmermann et al. (2005) created ROSE, which is a tool that finds coupling between different program components such as files, functions, or variables. ROSE can extract coupling information and predict future changes by mining software version histories. Since it uses version histories, ROSE can also find coupling between non-code items as well.

Ufuktepe and Tuglular (2021) developed Code Change Sniffer (CCS), which is a tool that predicts future code changes by utilizing static call information, forward slicing, and method change information. By combining these, CCS creates Markov chains to predict future method changes.

Our approach differs from previous works by combining MSR with dependency analysis using call graphs at the PR level, making it more suitable for modern code review processes. Unlike tools like DIVER (Cai and Santelices 2014) and CPCIA (Dai et al. 2022), which focus on method or statement-level changes, we provide a broader view by analyzing the entire PR, including associated files and changesets. MSR tools like ROSE (Zimmermann et al. 2005), on the other hand, focus on long-term trends in change propagation, rather than helping directly with real-time code reviews. In contrast, our approach is designed to analyze change impact within pull requests, aligning with modern practices like continuous integration (CI). Tools like FaultTracer (Zhang et al. 2012) and JRipples (Buckner et al. 2005) focus on static or dynamic analyses for fault localization and dependency tracking but don't use MSR data. By combining MSR and dependency analysis, our method gives developers real-time insights into the impact of changes during code review, making it practical for today's workflows.

2.2 Tools that Enhance Code Review Experience

A range of tools and studies have been created to improve various parts of the code review process, such as recommending reviewers (Yu et al. 2016; Hajari et al. 2024; Kazemi et al. 2022; Asthana et al. 2019), understanding and improving reviewer behavior (Chen et al. 2022; Egelman et al. 2020; Ebert et al. 2019; Mukhtarov et al. 2023), and assessing code quality (Chen et al. 2022; Chatley and Jones 2018; Balcı et al. 2021; Tuna et al. 2024). Research on code reviews has shed light on challenges that reviewers face, like finding relevant code sections, deciding how closely to examine code, and handling the complexity of working with multiple reviewers. These findings have helped shape tools that focus on specific parts of the review process to make it easier for reviewers to do their work.

Beyond these general improvements, some tools are specifically designed to make code reviews easier and give useful insights. Commercial static analysis tools such as SonarQube, Code Climate², PMD³, inform software practitioners of rule violations, potential security threats, and opportunities for code refactoring. These tools can also calculate various metrics like code coverage and technical debt to improve the quality of code reviews. In addition to static analysis tools, there are other commercial tools like Codium PR-Agent⁴ which utilizes large language models (LLMs) and GPTs to analyze PR diffs to enhance code reviews. These tools can summarize PRs, suggest code modifications, check test coverage, identify security vulnerabilities, estimate effort, and more.

Additionally, several academic tools are developed to enhance the code review experience.

Gasparini et al. (2021) developed ChangeViz, which is a tool to help developers understand and assess PR changesets. ChangeViz is integrated into the GitHub interface, allowing reviewers to navigate method calls and definitions without leaving the PR screen. By providing additional information about the changeset, it tries to help reviewers understand the contents of the reviewed PR.

Balcı et al. (2021) claimed that showing code changes in a visual form can be beneficial in code reviews. They implemented a tool called DERT, which shows the structure of the changes in a UML-like diagram called a relationship diagram. The diagram shows changed classes, methods, interfaces, etc., using different colors representing their change status like added, removed, or modified. In addition to these, this diagram shows how much a class changes after the proposed modifications. There is also an artifact map showing the relation between different code artifacts like commits, issues, code files, etc., which can be used to detect potential points of interest in code review.

Hijazi et al. (2021) proposed that biometric data like Heart Rate Variability (HRV) and eye movement dynamics can be used to detect which parts of the code are not well reviewed in the code review process. They implemented a tool called iReview, which collects these biometric data during code review and generates a code review evaluation report to the reviewer after the review is finished. This report indicates which parts of the code are not well-reviewed in the review process.

Barnett et al. (2015) claimed that code changes generally consist of loosely coupled changes like multiple feature additions, bug fixes, etc., which can make the review process harder. They developed a technique called CLUSTERCHANGES, which is a static analysis method that groups the correlated changes, decreasing the complexity of the review process.

Sharma and Sodhi (2019) presented a tool that utilizes the contents of a popular programming forum, StackOverflow, to enhance the code review process. The tool calculates the likelihood of the code piece being defective by finding code pieces that resemble the reviewed code on StackOverflow.

When compared to the above tools, CHID offers several unique contributions. Similar to commercial static analysis tools like SonarQube and Code Climate, CHID uses static analysis but stands out by deriving method dependencies for change impact analysis. While these tools help detect rule violations and potential refactorings, CHID helps reviewers understand the broader impact of changes on the codebase by identifying affected methods. CHID is also similar to Codium PR-Agent in that both tools aim to enhance the code review experience by augmenting pull requests. However, Codium PR-Agent primarily utilizes code diffs and LLMs to analyze PRs, while CHID focuses on history mining and static dependency analysis.

² <https://codeclimate.com>

³ <https://pmd.github.io>

⁴ <https://www.codium.ai/products/git-plugin>

Table 1 Details of metrics used in the approach

Metric	Description	Scope
Impact Graph	A graph that represents each function call of changed functions	Function
Bug Frequency	Ratio of number of bugs associated with a file	File
Code Churn	Amount of LOC that is added, deleted or modified	File
Co-Changed Files	Files that are modified together throughout same PRs	File
Author PR Merge Rate	Percentage of merged PRs of author	Author
PR Size	The amount of LOC included in a pull request	PR
Risk Score	Estimation of the likelihood of a pull request causing bugs or issues	-

This allows CHID to highlight potentially affected methods and provide insights into the broader impact of changes, which is not the focus of Codium.

In terms of visualization, CHID is similar to DERT (Balci et al. 2021), which represents changes in UML-like diagrams. Both tools visualize impacted areas, but CHID also calculates risk scores and shows method dependencies, allowing reviewers to assess both structural and functional implications. Compared to tools like CLUSTERCHANGES (Barnett et al. 2015), which group correlated changes to simplify the review process, CHID's method-level impact analysis offers a more precise view of potential effects. While Gasparini et al. (2021) allows navigation of PR changes, CHID complements this by offering actionable metrics like risk scoring and impact analysis.

With its focus on method dependencies, risk assessment, and visualizations, CHID aims to contribute meaningfully to the code review process and provide useful insights that can complement the features of other tools.

3 Our Approach

We started our research by conducting a requirement analysis of the current code review practices of practitioners. Since each individual can prioritize and care about different aspects of their code review practices (Sadowski et al. 2018; Kononenko et al. 2016), it was important to gather their ideas about how we could improve the current experience.

For this purpose, we conducted a feature feedback survey with 20 participants before finalizing the proposed approach⁵. Participants were selected from the authors' professional network through emails. Our criterion for selecting participants was to include individuals with diverse roles and experience levels, ensuring that the average amount of experience was above 7 years of experience. The invitation email includes brief information about the tool, the link to the survey and a deadline to finish the survey. The invitation did not offer any type of compensation for the participants. We initially contacted 25 practitioners and only 20 of them filled out the feature feedback survey. In the survey, we briefly introduced each metric (Table 1), asked if users would prefer it in risk score calculation, and asked for improvement suggestions. Also, we inquired about potential metrics for risk score calculation, their current CIA practices, and their emphasis on the CIA at the PR level.

⁵ The feature feedback survey questions are available in the replication package at <https://doi.org/10.6084/m9.figshare.27643755.v1>

Table 2 Feature survey participant demographics

Name	Total Experience (Years)	Position	Application Domain	Education Level
P1	10	Team lead	Home Appliances	BSc
P2	15	Manager	Home Appliances	MSc
P3	20	CEO	IoT platforms	MSc
P4	20	Manager	Defense systems	BSc
P5	20	Architect	Computer vision	MSc
P6	3	Software Developer	E-commerce	BSc
P7	2	Data Scientist	Banking	BSc
P8	2	Data Scientist	Banking	BSc
P9	20	Deputy of VP Engineering	Marketing	MSc
P10	11	Software Consultant	IT Services	MSc
P11	1	Data Scientist	Biotechnology Research	MSc
P12	15	Machine Learning Engineer	Banking	MSc
P13	18	Head of Security	Incident Management	BSc
P14	9	CEO	Engineering Intelligence	BSc
P15	1	Graduate Student	-	BSc
P16	1	Graduate Student	-	BSc
P17	20	Software Developer	Search Engine	MSc
P18	7	Senior Software Engineer	Telecommunications	BSc
P19	1	Graduate Student	-	BSc
P20	17	Principal Software Engineer	Tourism Services	MSc

Participants scored each proposed feature and gave their feedback related to these features. The demographics of feature survey participants can be seen in Table 2. Likert scale answer distribution for each metric can be seen in Fig. 1. Our metrics with their description and scope are given in Table 1. Each metric is explained in detail in its dedicated subsection under Section 3.1.

The following subsections present our approach, including our metrics, call graph algorithms, and risk score calculation. Implementation-specific details of our approach, such as used libraries and frameworks, will be discussed in Section 4.

3.1 Metrics

In this section, we present the definition, equation, scope, and category range values of the metrics included in the proposed approach. Each introduced metric, except missing co-change files, contributes to the risk score of the PR. After the metrics are calculated, their values are

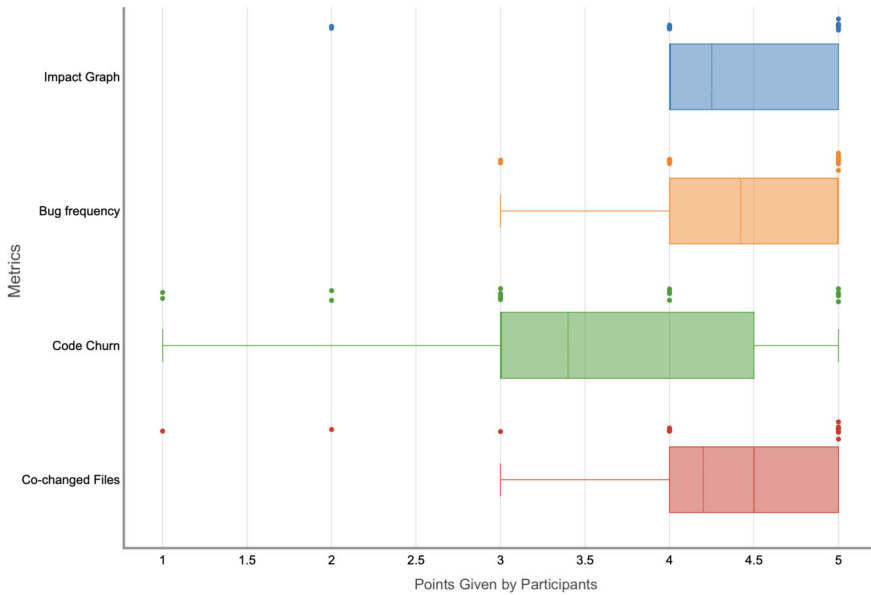


Fig. 1 Feature survey participants distribution by years of experience.

categorized using predetermined category ranges to normalize their contribution to the risk score. Code churn and bug frequency are file metrics, meaning they are calculated for each file. The proportion of files that exceed a threshold for these metrics, namely highly churned file ratio and highly buggy file ratio, contributes to the risk score. The category ranges and file metric thresholds have default values that may not be applicable universally, but practitioners can adjust these values based on their preferences. The default values of category ranges and file metric thresholds are detailed in related subsections.

3.1.1 Highly Churned File Ratio

Code churn is a measure of the amount of code change taking place within a software unit over time (Nagappan and Ball 2005). A study by Nagappan and Ball (2005) showed that certain code churn metrics correlate highly with flaws in the code. These metrics have a role in driving the development and test process, as highly churned code is more prone to defects and should be tested more carefully and thoroughly. The formulation of code churn of a file is expressed as shown in (1), where f_j is a file and LOC_{f_j} is a line of code in f_j . The nominator expresses the number of cumulatively changed (added, deleted, modified) lines of a code throughout the life of f_j and the denominator expresses the number of total lines of a code that f_j currently has. Code churn is a file-scoped metric, meaning it specifically considers the changes within the files of a PR. If the $Code\ Churn(f_j)$ is higher than five, f_j is considered as a highly churned file. The range values used to categorize highly churned file ratio in a PR, which is calculated by dividing the churned file amount by the total file amount in the PR, is given in Table 3.

$$Code\ Churn(f_j) = \frac{\text{Cumulatively Changed \# of } LOC_{f_j}}{\text{Total \# of } LOC_{f_j}} \quad (1)$$

Table 3 Category thresholds for highly churned file ratio in a PR

Category	Range Values
A	0
B	(0 - 0.16]
C	(0.16 - 0.33]
D	(0.33 - 0.50]
E	(0.5 - 1]

3.1.2 Highly Buggy File Ratio

A pull request is considered bug-related if it is labeled with a tag that includes the word *bug* or is linked to an issue with a similar label, as commonly found on platforms like GitHub (Liu et al. 2016). Files within these PRs are classified as bug-related files. The bug frequency of a file is defined as the ratio of its occurrences in the changesets of bug-related PRs to its total occurrences in the changesets of all PRs. This metric indicates the likelihood of a file causing a bug when modified in a PR depending on how frequently it has caused bugs in the past. Similar to code churn, bug frequency is also a file-scoped metric, meaning it specifically considers individual files within the context of pull requests. The calculation of a file's bug frequency is detailed in (2), where f_j is a file and PR_{f_j} is a pull request where f_j is in its changeset. If $\text{Bug Frequency}(f_j)$ is higher than 40%, the f_j is considered as a highly buggy file. The highly buggy file ratio of a PR represents the amount of highly buggy file presence in the changeset of the PR. The range values used to categorize the highly buggy file ratio, calculated by dividing the highly buggy file amount by the total file amount in a PR, is given in Table 4.

$$\text{Bug Frequency}(f_j) = \frac{\text{\# of Bug Related PR}_{f_j}}{\text{Total \# of PR}_{f_j}} \quad (2)$$

3.1.3 Missing Co-change Files

Co-change represents the evolutionary coupling between different files. When any two files frequently change together in version history, there is a high probability they will change together in the future again (Zimmermann et al. 2005). The formulation of the co-change rate of f_j with f_k is expressed as shown in (3), where f_j and f_k are different files, and PR_i is a pull request. Occ_{f_j, PR_i} is a binary variable that represents occurrence of the f_j in PR_i . Likewise, Occ_{f_k, PR_i} represents occurrence of the f_k in PR_i . The numerator represents the total number of PR_i where f_j and f_k are changed together, while the denominator represents the total number of PR_i where f_j changed. The co-change metric, which focuses on the

Table 4 Category thresholds for highly buggy file ratio in a PR

Category	Range Values
A	0
B	(0 - 0.12]
C	(0.12 - 0.2]
D	(0.2 - 0.3]
E	(0.3 - 1]

Table 5 Category thresholds for PR size

Category	Changed Lines of Code
A	[0, 10)
B	[10, 50)
C	[50, 200)
D	[200, 1000)
E	1000+

relationship between individual files within pull requests, marks pairs as co-changed when the $Co-changeRate_{f_j}(f_k)$ exceeds 50%. Since the co-change rate does not affect risk score calculation, it is not categorized. The co-change rate for each file pair is used to determine missing co-changed files in the changeset, which could be related to changes or affected by the changes. It prompts reviewers to check missing files that could be potentially related to the changes in the PR.

$$Co-change Rate_{f_j}(f_k) = \frac{\sum_{i=1}^n Occ_{f_j, PR_i} Occ_{f_k, PR_i}}{\sum_{i=1}^n Occ_{f_j, PR_i}} \quad (3)$$

3.1.4 PR Size

PR size is the number of changed lines of code in a PR. A large changeset can reduce the effectiveness of code reviews (Doğan and Tüzün 2022), thus increasing the risk factor. We used the size scale proposed by Doğan and Tüzün (2022) to categorize PR sizes accordingly, which is given in Table 5.

3.1.5 Author PR Merge Rate

Author PR merge rate corresponds to the ratio of the author's merged PRs to the total number of PRs opened by the author. As practitioners gain experience in the project, their PRs become more likely to be accepted (Dey and Mockus 2020; Legay et al. 2018). Since they have a better grasp of the project, intuitively, their PRs have a lower risk potential. The formulation for author PR merge rate is expressed as shown in (4), where a_j is a developer, PR_i is a pull request, PR_{i,a_j} is a PR_i created by a_j , and MPR_{i,a_j} is a merged PR_i opened by a_j . The author PR merge rate categorization is given in Table 6.

$$Author PR Merge Rate(a_j) = \frac{\sum_{i=1}^n MPR_{i,a_j}}{\sum_{i=1}^n PR_{i,a_j}} \quad (4)$$

Table 6 Category thresholds for author PR merge rate

Category	Range Values
A	[1 - 0.93)
B	[0.93 - 0.87)
C	[0.87 - 0.81)
D	[0.81 - 0.75)
E	[0.75 - 0]

Listing 1 An example Java class with four methods

```
public class Main {
    public void methodA () {
        methodB ();
        methodC ();
    }

    public void methodB () {
        methodD ();
    }

    public void methodC () {
    }

    public void methodD () {
    }
}
```

3.2 Call Graph

To find the function-level dependencies, we utilize a project-wide function call graph that shows the function call relationships in a codebase. Each function is represented as a node in the graph, and call relations are shown as directed edges. By using the call graph, developers can observe the function dependencies of their project. An example call graph for Listing 1 is given in Fig. 2. Since there are four function declarations and three function calls in the code, the generated call graph has four nodes and three edges.

3.2.1 Call Graph Generation

To create a call graph, we first generate AST for each file of the analyzed project. ASTs are hierarchical data structures that display the syntactic structure of source code, where each

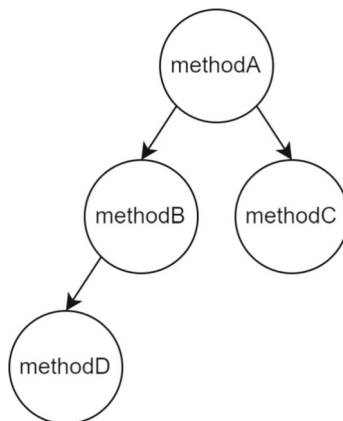


Fig. 2 Call graph generated from the Java code presented in Listing 1

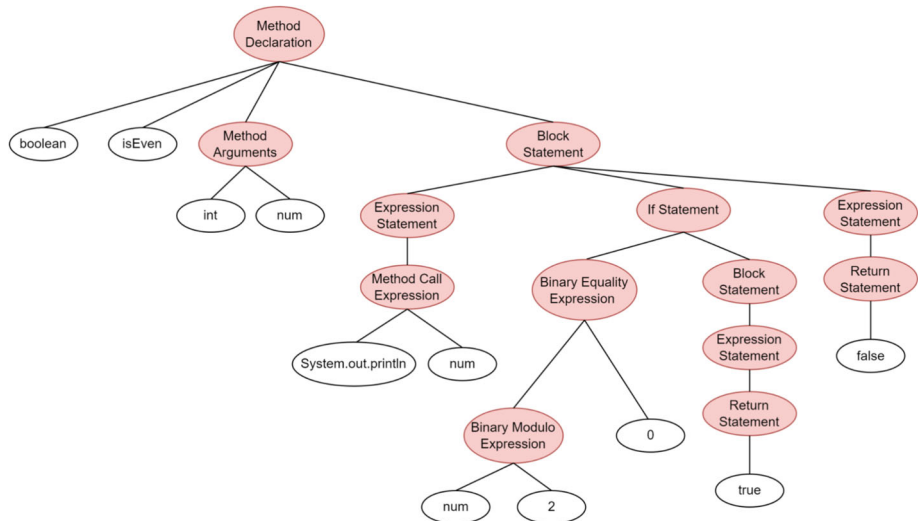


Fig. 3 AST representation of the Java code presented in Listing 2

node represents various programming language constructs such as expressions, statements, functions, etc. An example AST for Listing 2 is given in Fig. 3. After creating and parsing ASTs for each project file, we can extract function declaration and call information, which is the core information needed in the call graph creation. However, since ASTs generally do not have type-related information, it is not possible to derive which class the function calls belong to. Thus, we utilized a type resolver tool to fetch this information. Call graph can be generated by combining AST and the derived type information. Figure 4 shows the general call graph creation process.

3.2.2 Call Graph Update

Although the call graph is useful for uncovering function-level dependencies, it becomes outdated with each accepted code. To keep the graph up-to-date, it needs to be updated regularly. Since call graph generation is time-consuming for large scale projects, we propose an algorithm to update the existing project call graph after a PR is merged. To do this, we need to extract added/removed function calls and definitions. We first generate two call graphs, one for the old and one for the new version of the changed files. Since these call graphs are created solely from the changed files, the process takes significantly less time compared to

Listing 2 A sample Java method to check if a number is even

```

public boolean isEven(int num) {
    System.out.println(num);
    if (num % 2 == 0) {
        return true;
    }
    return false;
}

```

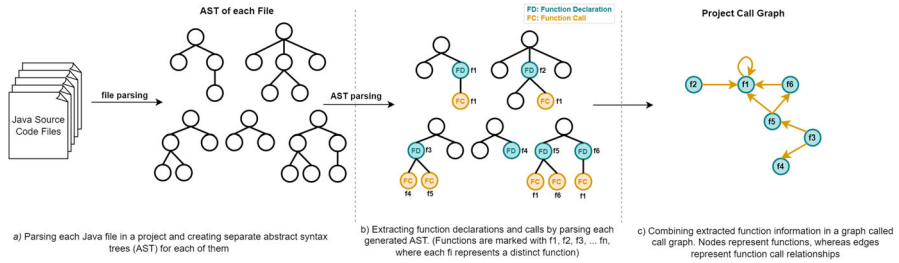


Fig. 4 Call graph generation process

using the entire project. After creating these graphs, we compare them to find newly added or removed function calls and definitions. The pseudocode of the call graph update algorithm is presented in Algorithm 1, while Fig. 5 depicts the steps of the process.

Algorithm 1 Update call graph with pull request changes.

```

1: procedure UPDATECALLGRAPH(callGraph, pullRequest)
2:   oldFilePaths  $\leftarrow$  create an empty list
3:   newFilePaths  $\leftarrow$  create an empty list
4:   for each changedFile in pullRequest.changedFiles do
5:     filePath  $\leftarrow$  get file path from changedFile
6:     status  $\leftarrow$  get status from changedFile
7:     if status is ADDED then
8:       add filePath to newFilePaths
9:     else if status is REMOVED then
10:      add filePath to oldFilePaths
11:     else if status is MODIFIED then
12:      add filePath to newFilePaths
13:      add filePath to oldFilePaths
14:     end if
15:   end for
16:   execute git checkout pullRequest.destination
17:   oldCallGraph  $\leftarrow$  createCallGraph(oldFilePaths)
18:   execute git checkout pullRequest.origin
19:   newCallGraph  $\leftarrow$  createCallGraph(newFilePaths)
20:   modifications  $\leftarrow$  extract added and deleted method calls and definitions
21:   update call graph using extracted modifications
22:   return true
23: end procedure

```

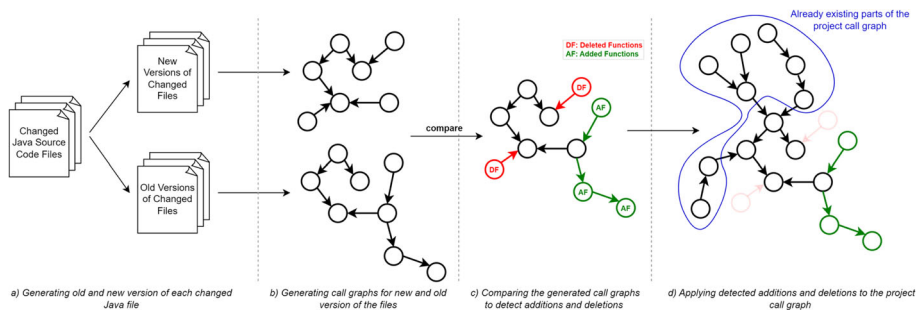


Fig. 5 Call graph update process



When the project call graph is up-to-date, it can be used to detect the potential impact of changes in a PR. In our impact definition, we declare methods invoking any changed methods as potentially affected. However, if any of the affected methods are also called inside of another function, the change impact can also ripple to such functions. We first extract the changed methods from the changeset by generating two sets of ASTs, one for the old and one for the new version of the changed files. After these ASTs are generated, we extract the changes using the tree differencing algorithm presented by Fluri et al. (2007).

3.2.4 Impact Size

a) Generating old and new version of each changed Java file

b) Generating ASTs for new and old version of the files

c) Applying tree differencing algorithm on the generated ASTs to detect the modified functions

d) Finding potentially impacted functions by checking up to third degree neighbours of changed functions in the pre-call graph

 Springer

Algorithm 2 Find potential impact of a changeset.

```

1: procedure FINDPOTENTIALIMPACT(changedFiles, callGraph)
2:   changeSet  $\leftarrow$  create an empty set to store changes
3:   for each changedFile in changedFiles do
4:     if changedFile status is ADDED then
5:       continue to the next iteration
6:     end if
7:     shaOld  $\leftarrow$  get commit hash value of the old version of the changedFile
8:     shaNew  $\leftarrow$  get commit hash value of the new version of the changedFile
9:     oldFile  $\leftarrow$  retrieve old file using git show shaOld command
10:    if changedFile status is removed then
11:      newFile  $\leftarrow$  create an empty file
12:    else
13:      newFile  $\leftarrow$  retrieve new file using git show newSha command
14:    end if
15:    changes  $\leftarrow$  extract classified source code changes between oldFile and newFile
      using ChangeDistiller
16:    for each change in changes do
17:      if change is related to a method then
18:        add change to changeSet
19:      else if change is related to a class and is a method removal then
20:        add change to changeSet
21:      end if
22:    end for
23:  end for
24:  impactedMethods  $\leftarrow$  getThirdDegreeNeighbors(changeSet, callGraph)
25:  return changeSet, impactedMethods
26: end procedure

```

methods in a pull request, calculating their broader effect within the project. To calculate the impact size, we execute a PageRank algorithm on the whole project call graph. PageRank is a link-based algorithm used to assign numeric weights to collection elements based on their relative importance (Brin and Page 1998). In a graph, if a node $node_1$ has a higher PageRank score than another node $node_2$, it indicates that $node_1$ holds a greater significance than $node_2$ in terms of centrality. When calculating this significance, the PageRank algorithm does not just examine the incoming edges but also considers incoming edges to the neighboring nodes recursively. The formulation for impact size is expressed as shown in (5), where N_i is a set of all methods in the project when PR_i is opened, C_i is a set of changed methods in PR_i , and $Pagerank(v_i)$ is PageRank score of method v_i in the project call graph when PR_i is opened. The categorization of impact size is given in Table 7.

$$\text{Impact Size}(PR_i) = \frac{\sum_{v_i \in C_i} \text{Pagerank}(v_i)}{\sum_{v_i \in N_i} \text{Pagerank}(v_i)} \quad (5)$$

Table 7 Category thresholds for impact size

Category	Range Values
A	[0 - 0.01)
B	[0.01 - 0.03)
C	[0.03 - 0.06)
D	[0.06 - 0.08)
E	[0.08 - 1]

3.3 Risk Score

After calculating previous metrics, we calculate an overall risk score for the PR, indicating how risky the proposed changes are. In the literature, there are different risk concepts such as post-release defect risk (Krutauz et al. 2020; Thongtanunam et al. 2015), and risk of knowledge loss (Hajari et al. 2024; Kazemi et al. 2022), among others. Our risk score is similar to post-release defect risk, focusing on the potential risks introduced by the proposed changes. We formulated this score as a weighted sum of our metrics, given in Fig. 8. As stated above, the corresponding metric categories (A to E) are converted to numeric values (1 to 5), multiplied by the given weights in Fig. 8, and incorporated into the risk score formula. Users can modify the metric weights in the risk score formula according to their project's specific needs.

The rationale for the selected metric coefficients is rooted in the significance of certain metrics in contributing to overall risk. For instance, for a ripple effect to occur, a change in the PR must impact other parts of the codebase. Any other metric value does not impose as much importance if the change effect does not propagate throughout the codebase. Therefore, we decided that impact size should have the highest contribution to the risk score. Conversely, even if a PR contains issues at the beginning, it is often merged after revisions and reviews. Over time, developers working on the same project tend to have similar merge rates. However, we do not want to entirely exclude this metric, since a newcomer (or a contributor to an open-source project) may have a lower rate of accepted PRs at a particular time. Thus, the author's PR merge rate receives the lowest contribution rate among the metrics, while the other three metrics have relatively similar contributions to the overall risk score calculation.

4 Implementation

In this section, we discuss details of our CIA implementation. The following subsections introduce the selected background technologies and how we utilized them in our call graph server, GitHub bot, and web application.

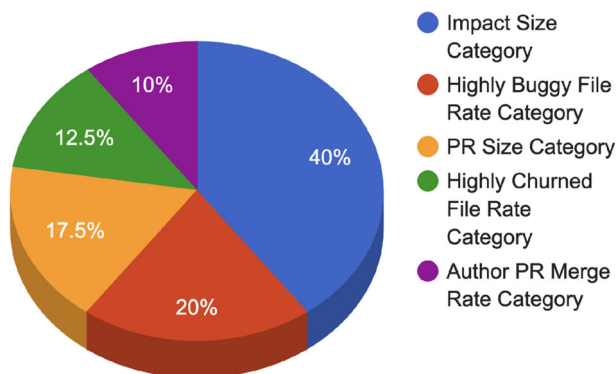


Fig. 8 Current risk score formulation

4.1 Call Graph Server

The call graph server is a microservice for executing call graph-related tasks outlined in Section 3 such as call graph creation/update, impact calculation, etc. The server is implemented using Java and Spring Boot. Currently, it can analyze projects that use Java 8.

In the call graph generator algorithm, we used JavaParser⁶ for AST generation. It has type resolver functionality and supports Java 8, which is suitable for our application. After generating the call graph, we store it in a graph database called Neo4j⁷. We chose Neo4j over non-graph databases because it offers several built-in graph algorithms and simplifies working with graph data.

For tree differencing algorithm implementation, we used an open-source library called ChangeDistiller, which is provided by Fluri et al. (2007).

4.2 GitHub Bot

The GitHub bot is the primary analysis controller of the CHID, which executes initial and incremental analysis. The initial analysis is started after the bot installation, whereas incremental analysis is carried out with each new PR. The workflow of the incremental analysis is given in Fig. 9. GitHub bot is implemented using Javascript and Probot framework⁸ which simplifies the process of creating a GitHub App⁹. This GitHub App can interact with repositories, issues, PRs, and other GitHub features.

4.2.1 Initial Analysis

As stated, initial analysis is executed only once when the GitHub bot is installed in a repository. Firstly, general information about the repository, default branch, PRs, and contributors are fetched using GitHub API. Concurrently, project call graph creation is started. Then, the author PR merge rate for each contributor is determined. After that, code churn, bug frequency, and co-change values are calculated for changed files in the PRs respecting the following criteria:

1. Either open or merged
2. Has at least one Java file
3. The PR's base branch is the repository's default branch.

Each of the metrics calculated in this analysis is stored in a MongoDB instance.

4.2.2 Incremental Analysis

Incremental analysis is performed for each open PR pointing to the default branch. Historical metrics are updated according to the changes in the PRs that have been merged into the default branch. However, our analysis does not include newly created files, focusing only on modified

⁶ <https://javaparser.org>

⁷ <https://neo4j.com>

⁸ <https://probot.github.io>

⁹ <https://docs.github.com/en/apps/using-github-apps/about-using-github-apps>

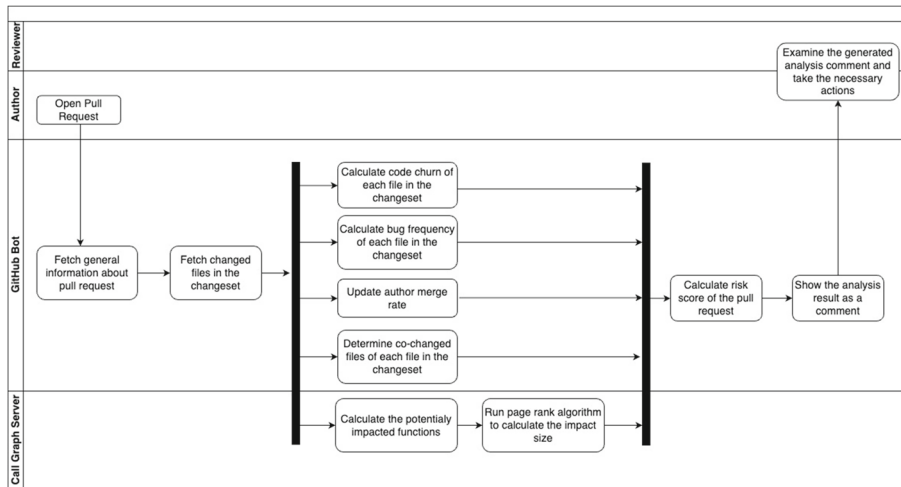


Fig. 9 Activity diagram of the analysis steps

or existing ones. After completing the analysis, a comment is attached under the analyzed PR to show the results. An example analysis can be seen in Fig. 10.

For each file in the new PR's changeset, like the initial analysis, we calculate the code churn, bug frequency, and co-change. Then leveraging this information, we determine the highly churned file count, highly buggy file count and missing co-changed file amount in the changeset of the PR. After these steps are done, we send a request to the call graph server to calculate the potentially impacted methods and the impact size of the PR. After these are calculated, we evaluate the risk score of the PR.

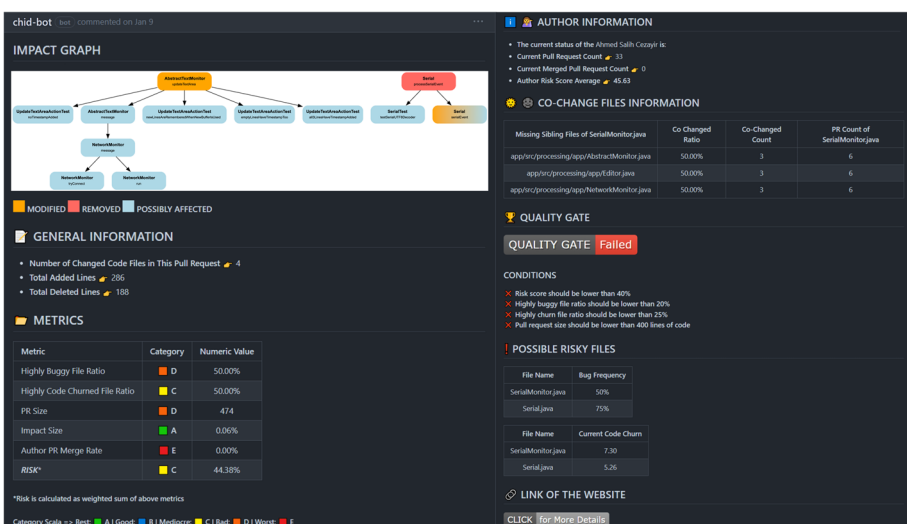


Fig. 10 Example analysis output

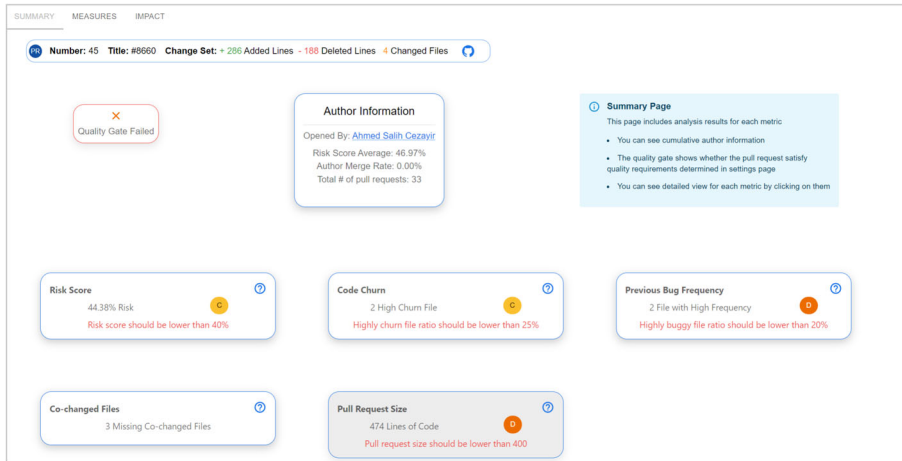


Fig. 11 Analysis summary page of web application

4.3 Web Application

The web application is a single-page app where all the analyses are displayed in detail, shown in Fig. 11. Similar to SonarQube, users can see the category value of their metrics and quality gate results which can be configured based on their selections. They can see reasons for a quality gate failure beneath the corresponding metrics and take action based on the results. Based on their DevOps practices, users can add this information to their pipeline to strictly force the quality gate constraints. Users can configure analysis metrics and thresholds using this application, as shown in Fig. 12. The front-end part is developed using React and JavaScript. On the server side of the application, JavaScript, Node.js, and Express.js are used

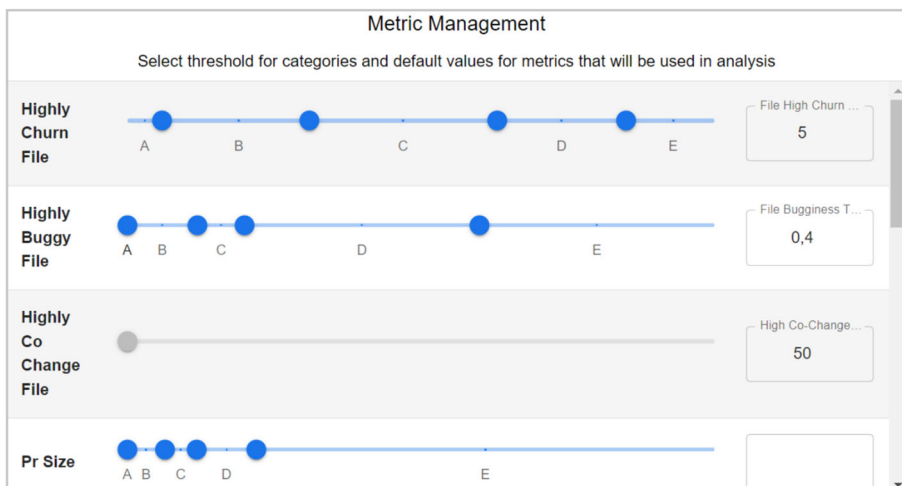


Fig. 12 Configuration screen of web application

to develop a REST API. The data is fetched from the MongoDB instead of performing any significant calculation or using GitHub API.

5 Focus Group Study

Focus group is a research method where data is collected through group discussion concentrated on a research topic. This methodology allows researchers to collect participants' perspectives about the particular area of study (Kontio et al. 2008). We conducted a focus group study to get practitioners' perspectives on the CHID's effectiveness in improving the code review process. In this section, we present the methodology and design of the focus group study. Figure 13 summarizes the focus group study workflow.

5.1 Research Objectives and Questions

The Goal-Question-Metric (GQM) framework is used for effective measurement of organizational projects focused on certain *goals* that a project should attain, *questions* that define the way for goal attainment, and *metrics* that supply the data to answer these questions (Caldiera and Rombach 1994). This study's objective is formulated according to the GQM method given as follows: “**Analyze** proposed CIA approach **for the purpose** of evaluation **with respect to** the effectiveness, and applicability of the proposed CIA approach; and the effectiveness of the proposed risk score approach **from the point of view of** software practitioners.”

Based on the formulated goal, we have determined the following research questions:

- **RQ1. How effective is analyzing change impact on the PR level for enhancing the code review experience?**

We will focus on the perceived effectiveness and computational feasibility of CHID in improving the code review experience on the PR level from different aspects.

- **RQ2. How effective is the introduced risk score approach for indicating the change impact of a PR?**

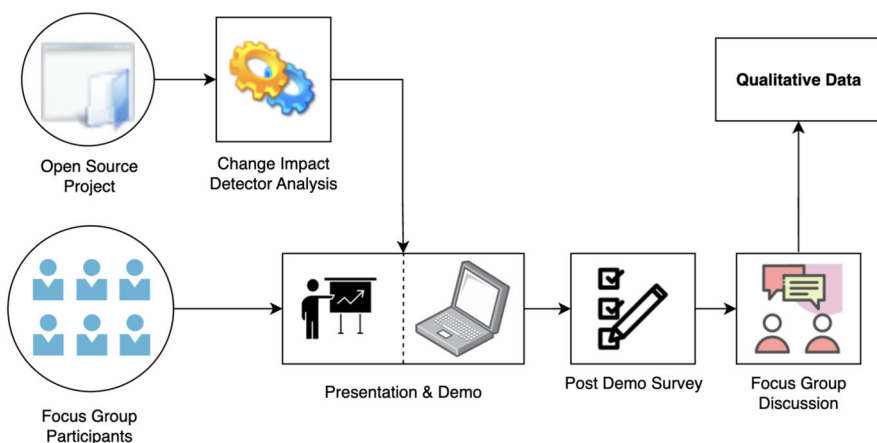


Fig. 13 Focus group methodology

- **RQ2.1 Does the introduced risk formula correctly represent the change impact of a PR?**
- **RQ2.2 Does the risk score representation thoroughly reflect the change impact of a PR?**

We will focus on whether the proposed risk score approach can express the risk perception of a PR.

5.2 Design

Here, we describe the design of our focus group session.

5.2.1 Preparatory Work

Prior to conducting the focus group study, we held live demo sessions with four software engineers in one-on-one settings. During these demos, we demonstrated the tool, gathered participant feedback, and addressed their questions and concerns. These insights, combined with our reflections on the topic, influenced the focus group outline, as well as the questions we asked during the focus group discussions and the post-demo survey¹⁰. In preparation for the focus group sessions, we also created a presentation to introduce the tool, a post-demo survey aligned with our research questions, and a tool video¹¹ to familiarize participants with the tool's features. This video was sent along with the focus group study invitations. Additionally, we prepared a post demo survey to get participants' written feedback on the approach. Furthermore, to provide a practical context for the discussions, we forked the Arduino project¹² and selected an open pull request¹³ for analysis using our default metric category and threshold values.

5.2.2 Participant Selection and Session Management

To select participants, we used a hybrid approach of purposeful sampling (Palinkas et al. 2013) and convenience sampling (Stratton 2021). We aimed to include a diverse range of individuals based on their years of experience and the industries of their companies. In addition, we wanted our participants to be eager and willing to participate our study and discussions. We designed our focus group study to be 150 minutes long face to face sessions, to get the most coverage from practitioners' perspectives. Since it required time investment from attending practitioners and we wanted the focus group to be more interactive, we selected participants accordingly. To overcome connectivity issues and ensure participant engagement, we conducted the study in person. We initially invited 12 practitioners from our professional network to the study, where only seven of them accepted to attend. Participants had varying roles, including developers, architects, and project managers. The participant demographics are given in Table 8.

During the study, co-moderators (the first and second author) were present with the participants. Both authors are involved in topic coverage and present the material interchangeably.

¹⁰ The focus group outline and the asked questions are shared in the replication package at <https://doi.org/10.6084/m9.figshare.27643755.v1>

¹¹ <https://youtu.be/7j7WkA1RIsE>

¹² <https://github.com/arduino/Arduino>

¹³ <https://github.com/arduino/Arduino/pull/8660>

Table 8 Focus group participant demographics

Participant	Role	Experience In Current Position In Years	Total Experience In Years	Code Review Experience In Years	Application Domain	Education Level	Programming Language Expertise
P1	Software Engineer	20	30	28	Command and control systems	eMBA	Java, Python
P2	Software Engineer	1	2	2	Software analytics	MSc	Go, Python
P3	Software Developer	2	3	3	Cybersecurity	BSc	Java, Python
P4	Architect	2	20	12	Computer vision	MSc	C#, Java
P5	Project Manager	5	24	12	Management information systems	BSc	Java, Python
P6	Software Engineer	3	4	4	Healthcare	MSc	Kotlin, Java
P7	Software Engineer	3	3	3	Real estate operations	MSc	PHP, Javascript

Participants asked questions related to our approach and expressed their perspectives about the potential effects of the CHID on the code review process, the perceived effectiveness of risk score formulation, and possible improvements for the tool based on the detailed presentation and tool demo. Each session is recorded in both audio and video formats with the participants' consent. We used two voice recording devices for audio recordings. We utilized the Zoom application on a computer for video recording. Each session started with a brief explanation of the case study's goal, background information related to the study, and the data usage disclaimer.

5.2.3 Background Presentation and Tool Demo

At the beginning of the sessions, we briefly explained the survey questions to our participants so that they could concentrate on key points throughout our presentation. Given that each session is planned to last 150 minutes, it helped our participants to preserve their energy throughout the session. Then, we highlighted the definition, importance, and calculation method of our metrics. Afterward, we conducted a live demo using the selected PR for our participants. We explained the analysis results and the background mechanism of the tool. During the presentation and demo, participants had the opportunity to ask questions about the tool and our approach.

5.2.4 Post Demo Survey

Before the main discussion phase of the focus group, participants filled out the post-demo survey in our observation. We answered their questions throughout this part. This survey consisted of 22 questions, which could be completed in around 20 minutes, including open-ended, Likert-scale, and multiple-choice questions about demographics, CHID's effect on code reviews, and the risk score calculation mechanism.

5.2.5 Focus Group Discussion

According to the guidelines of Krueger and Casey (2014), we created an outline of focus group discussion. As an opening question, we asked participants to describe their companies' code review and CIA processes. Then, as a transition question, we questioned their general impressions about the tool. Afterward, as key questions, we asked for their opinion regarding the CIA on the PR level and asked them to elaborate on their answers to survey questions.

5.3 Qualitative Data Analysis

After the focus group sessions, audio recordings are transcribed and translated into English without using any programs. We did not use video recording since we were able to discern participants from audio recordings. We utilized the open coding phase of grounded theory (Stol et al. 2016). During the first iteration, the first and second authors independently conducted open coding on the transcribed audio recordings. They identified 32 and 20 codes, respectively. While some codes were labeled differently, they referred to similar underlying concepts. For instance, the first author used the code *authenticity* to describe sections discussing the originality of our approach, whereas the second author used *novelty* for the same concept. Thirteen such instances of differing labels for the same concept were found.

Table 9 Data collection methods per research question

Collection Method	RQ1	RQ2.1	RQ2.2
Focus Group	✓	✓	✓
Post-Demo Survey	Q5, Q6, Q7 Q8, Q9, Q10		Q13, Q22
Computational Feasibility Experiments	✓		

Additionally, three codes—*speed*, *static analysis*, and *dynamic analysis*—were identical between the authors.

There were 21 codes that differed, either because one author did not code a specific concept or used a broader code that encompassed the other's. After this first iteration, the first and second authors came together in the second iteration to merge their codes and agreed upon a preliminary set of 35 codes¹⁴. In the third iteration, all three authors convened to review and revise the codes, addressing any unmarked or incorrectly coded sections. When disputes arose, the third author facilitated the resolution of these differences.

6 Results

In this section, we present the results for each research question, with a focus on participants' perceptions from the tool demonstration, alongside the quantitative data gathered from experiments on three open-source projects. To ensure a comprehensive analysis, we employed a variety of data collection methods tailored to each research question. Table 9 summarizes these methods, including focus groups, post-demo surveys, and computational feasibility experiments, highlighting which methods were used to address each research question. Detailed results will be discussed in the following sections.

6.1 Effectiveness of CIA on PR Level (RQ1)

We evaluated the perceived effectiveness of our approach through computational feasibility experiments, along with post-demo survey questions and focus group discussions. The responses to these questions are shown in Figs. 14 and 15. The distribution of individual answers created by mapping the participants' answers to Likert scale questions into a numeric scale from 1 to 5 is visualized in Fig. 15 and combined in Fig. 16.

6.1.1 Facilitate Code Review Process

Participants evaluated whether CHID can improve the code review experience in general. We asked, "Do you think the Change Impact Detector is beneficial for the code review process?" The results are given in Fig. 14. Participants rated an average of 4 out of 5 for the tool's capability of providing benefits in the code review process.

Participant 1 (P1) stated, "There will be an increase in overall effectiveness and acceleration when this tool is used." P2 opposed this statement and said, "The results need a senior's review. Let's say an architectural decision was taken because of a requirement that can create potential problems. I think it does not make my review process 100% efficient because the

¹⁴ Codes are shared in the replication package at <https://doi.org/10.6084/m9.figshare.27643755.v1>

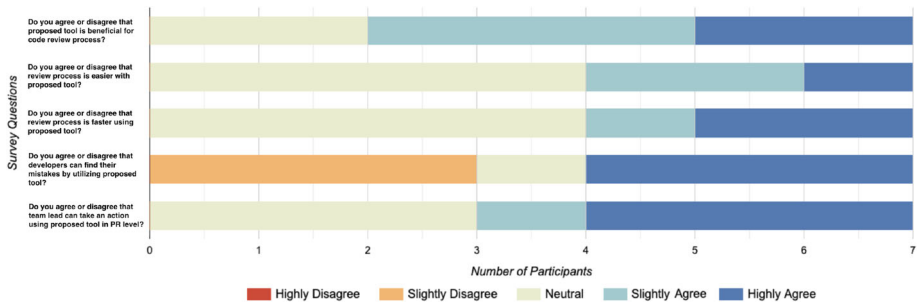


Fig. 14 Participants answers to Likert scale questions

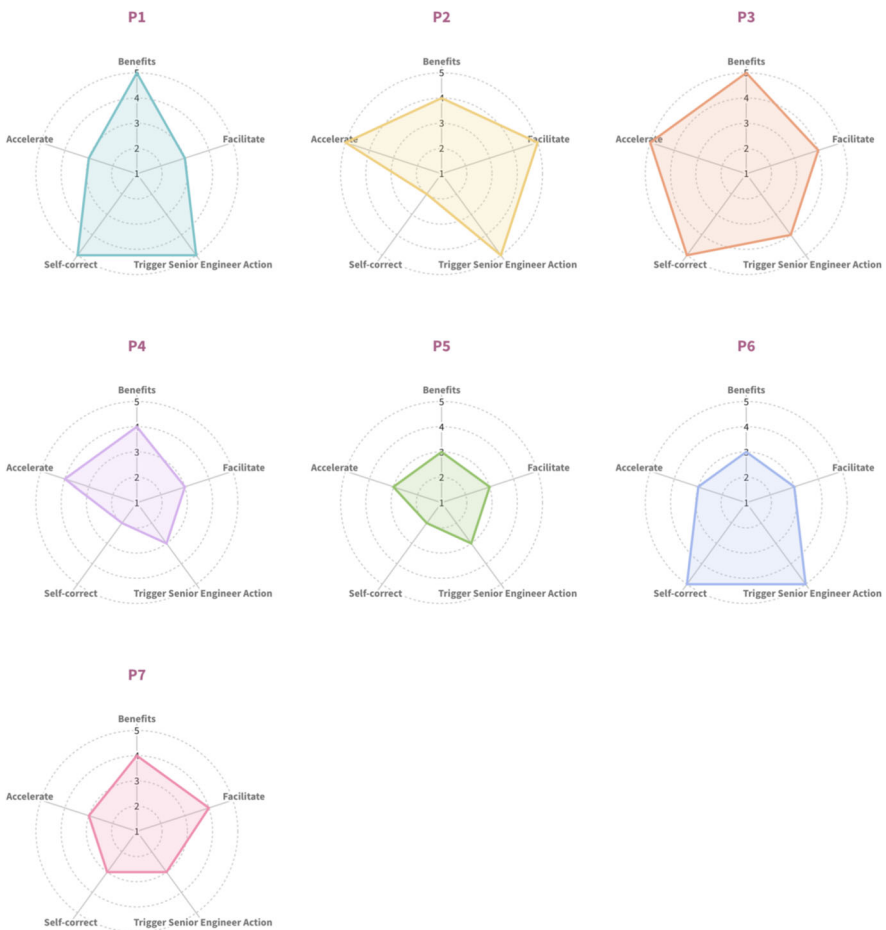


Fig. 15 Mapped Likert scale answers for each participant. Each axis illustrates a different aspect that affects the perceived effectiveness. (1) *Benefits* represents the potential of CHID to improve the code review experience. (2) *Facilitate* and (3) *Accelerate* depicts facilitation and acceleration effect of the tool. (4) *Self-correct* reflects the tool's potential to help developers find their own mistakes. Lastly, (5) *Trigger Senior Engineer Action* reflects the tool's potential to guide other team members to take action leveraging results

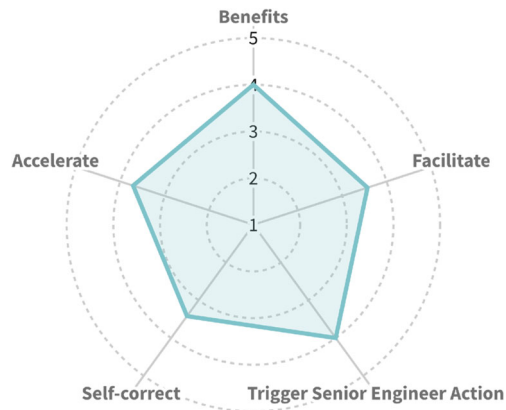


Fig. 16 Mapped Likert scale answers combined

tool is unaware of the underlying architectural problem.” They also discussed the facilitation of the code review process. We asked, “Do you agree or disagree that the review process is easier with Change Impact Detector?” The results are given in Fig. 14. According to the results, participants rated an average of 3.6 out of 5 for the facilitation effect of the tool on the code review process. P2 indicated, “The tool adds something to the system but does not automate my code review process. I still need to review it manually. It does not 100% measure that the changes in the opened PR cause anything bad.” Also, some participants implied that there is a learning curve for the tool. P1 emphasized, “The job would become more complex as the amount of things I have to look at has increased... Even after I learn the tool, there will still be more things I need to do. So the number of things to look at would increase.”

6.1.2 Accelerate Code Review Process

Participants assessed whether CHID makes the code review and development process faster. We asked, “Do you agree or disagree that the review process is faster using Change Impact Detector?” The results are given in Fig. 14. Participants rated an average of 3.7 out of 5 for the tool’s expediting effect on the code review process. There were opposing ideas related to this perspective. P5 said, “It is not possible for me to reduce the amount of time I need to complete the review using only your tool.” There were also some neutral perspectives. P1 highlighted, “I could not say the tool could made the review process faster because the number of things to look at increased.” P2 agreed with P1 and stated, “There is more information to look at directly when using this tool. Therefore, it creates manual work instead of speeding it up.” Some participants thought that interpreting impact graphs is not easy, especially for developers who are not familiar with the tool. P3 stated, “For example, looking at the graph takes time. I think I will be comfortable over time. Some of the presented data are just numeric, which is easy to understand, but the visual part takes time.”

6.1.3 Take Action

Participants evaluated whether users can take action after checking CHID’s analysis. We asked, “Do you agree or disagree that developers can find their mistakes by utilizing Change Impact Detector?” The results are given in Fig. 14. Participants rated an average of 3.4 out

of 5 for the tool's impact on assisting developers in finding their errors. They had contrasting opinions on this question. P3 stated, "If the analysis tells the PR's risk score is high, more tests can be added." Contrarily, P4 said, "I do not know if I can get anything out by just interpreting the graph. I did not totally get the proposed value with this feature."

Apart from this, we wanted to see whether senior team members (other than reviewer and developer) can utilize our analysis. We asked, "Do you agree or disagree that a team lead/senior engineer can take action using the Change Impact Detector at the pull request level?" The results are given in Fig. 14. Participants rated an average of 3.7 out of 5 for the tool's capability of assisting team leads in taking action using the results. They thought our impact graph might help find overlooked methods. P6 denoted, "Who will find a method that calls another method which calls a deleted method? You will have to trust the tests for this instance." Also, many other use cases are identified throughout the discussion. P7 highlighted, "Sometimes the product manager wants a feature but the developer wants to refactor. You have to give him a reason. If they see it in the risk factor, it will be a use case for developers." Also, P7 stated another use case, "I can use impact graph to learn the system or say to my lead that I will delete a method by showing its relations from impact graph."

6.1.4 Granularity Level

Participants had varying ideas about the optimum granularity level of the analysis. P1 highlighted the proposed tool should be able to respond to different development approaches like trunk-based development (Forsgren et al. 2017). P2 shared that the CIA would be more applicable to them if it focused on commits since they review each commit separately in their company. P3 said, "Since we work on a pull request basis, it would be suitable for us." P4 stated that the approach should focus on the project requirements instead of focusing solely on PRs. P7 denoted, "Beyond the PR basis, I'd like to see how risky the file I'm working on is."

6.1.5 Computational Feasibility

To demonstrate the practical applicability of our tool, we needed to verify that it offers reasonable performance and speed. To evaluate these criteria, we carried out several experiments on different-sized projects, namely *joda-time*¹⁵, *OpenPDF*¹⁶, and *Arduino*¹⁷.

The main criteria for selecting projects were the following:

1. The project uses GitHub for version control and issue tracking.
2. The project is primarily written in Java 8, accounting for at least 70% of the repository's code.
3. The project has at least 1,000 stars.
4. The project has at least 300 pull requests, including both opened and closed ones.

In addition, when selecting projects, we ensured they varied in size, allowing us to evaluate the performance of our tool across different-sized codebases. Detailed information about the selected projects is given in Table 10.

To evaluate the performance of our tool, we first measured the time required to generate the initial call graph and perform the initial project analysis. The results are given in Table 11.

¹⁵ <https://github.com/JodaOrg/joda-time>

¹⁶ <https://github.com/LibrePDF/OpenPDF>

¹⁷ <https://github.com/arduino/Arduino>

Table 10 Properties of the selected open-source projects

Project	Pull Request	File	Line of Code	GitHub Star	Contributor
joda-time	335	454	98468	4972	91
OpenPDF	666	888	118917	3489	122
Arduino	1499	684	188740	14111	249

Joda-time takes the longest time to generate the call graph, with 8.3 minutes, which is expected given the high number of nodes (function declarations) and edges (function calls) in its call graph. In contrast, Arduino achieves the shortest call graph generation time at 1.3 minutes, reflecting its smaller scale. For the initial analysis, OpenPDF requires the most time at 28.4 minutes, likely due to its larger number of files and pull requests. Conversely, Joda-time has the shortest initial analysis time of 12.6 minutes, benefiting from fewer files and pull requests.

In addition to these, we measured the execution time of incremental analysis on pull requests with different changeset sizes. To reflect the real-life scenarios, we defined the following different sample changeset configurations for our experiments:

1. **Single Method in One File:** Changes involve only one method within a single file.
2. **Five Methods in One File:** Changes involve five methods, all within a single file.
3. **Five Methods in Each of Five Files:** Changes involve 25 methods, with five methods in each of five separate files.

In each scenario, the methods and files were selected randomly to better reflect typical changeset variations. We performed the analysis multiple times to capture a range of execution times. Specifically, we executed each scenario five times with varying inputs to account for variability and ensure the accuracy of our results. The average execution times, calculated from these multiple runs, are presented in Table 12. The results show that, as the number of methods and the complexity of the changeset increase, so does the execution time. OpenPDF exhibits the highest execution times in all changeset sizes, likely due to its larger call graph and number of files, while joda-time shows the lowest execution times, reflecting its relatively simpler structure. In the largest changeset case, OpenPDF takes 22.4 seconds to complete the analysis, while Arduino and joda-time take 16.7 and 12.6 seconds, respectively.

6.2 Effectiveness of Formulating a Risk Score (RQ2)

Throughout the discussion, participants expressed that when a PR has a high risk score, a reviewer would examine it more carefully. By utilizing additional information provided by CHID, reviewers can make informed decisions, making the review process more effective. In addition, P7 stated that it would enable reviewers to focus more on important PRs, considering

Table 11 The initial analysis and call graph generation execution times (in minutes)

Project	Nodes	Edges	Call Graph Generation Time	Initial Analysis Time
joda-time	9330	25930	8.36	11.76
OpenPDF	6342	14879	3.87	28.37
Arduino	2124	3355	1.25	25.67

Table 12 The average incremental analysis execution time (in seconds) for different changeset sizes

Project	1 Method, 1 File	5 Methods, 1 File	5 Methods, 5 Files
joda-time	7.38	7.46	12.58
OpenPDF	16.83	17.10	22.43
Arduino	11.79	12.35	16.65

their already limited availability. To evaluate the perceived effectiveness of formulating a risk score in depth, we also examined two main aspects: the adequacy of its representation and the correctness of its formula.

6.2.1 Adequacy of Risk Score Representation

Participants are asked about the current risk score representation. Although the majority of the participants said risk score as a single numeric value could be useful, some of them also stated that displaying it in a non-numeric format might be more beneficial. P4 stated, “Using a radar chart instead of a single numeric risk score could be more helpful. Reducing it to a single number may cause losses in details.” In addition, P1 stated that showing a numeric value instead of the corresponding category might confuse developers. They stated, “Broader categorizations such as high and low can be made. The risk score can be shown as high, low, or medium. Maybe just show categories instead of numbers.”

Overall, radar chart-like visualizations were among the most widely supported suggestions from participants, as many felt they would improve clarity and flexibility in representing risk scores.

6.2.2 Correctness of Proposed Risk Score Formulation

Participants are asked whether the proposed risk score formulation correctly represents the impact of a PR. They also highlighted some improvement points. In the post-demo survey, participants gave an average of 3.2 out of 5 for the risk formula correctly representing the change impact of a PR. The majority of them agreed that metrics used in the risk formula are useful to represent a risk. However, some of the participants disagreed with the use of the author PR merge rate. P7 stated, “I am not sure whether there is any PR opened to the master branch, which is closed without merging. It may stay open for 1 or 2 weeks but eventually merges.” Other participants also agreed with P7, saying this metric is more suited for OSS projects. In addition to these, some participants said that some practices they follow might disturb the calculation of this metric. P6 stated, “Sometimes I open a PR to use CI/CD since I do not want to run all the tests locally. I am making a change and opening a PR just to make sure it works. So, this may not be valid for some people.” We asked participants which metrics would be useful to add when designing the risk score. The results are given in Fig. 17. For the risk formula, the agreement ratio of our proposed metrics with the focus group participants’ metric choices is 71%. However, according to participants’ choices, the risk formula might be expanded to include several more metrics like file inheritance or code complexity. These choices also created the basis for our metric weights given in Fig. 8

In addition to metrics used in the risk formula, we also asked about the thresholds and coefficients used with metrics. The majority of the participants said that default values of these need further justification; however, they also said this is not a big concern since they are

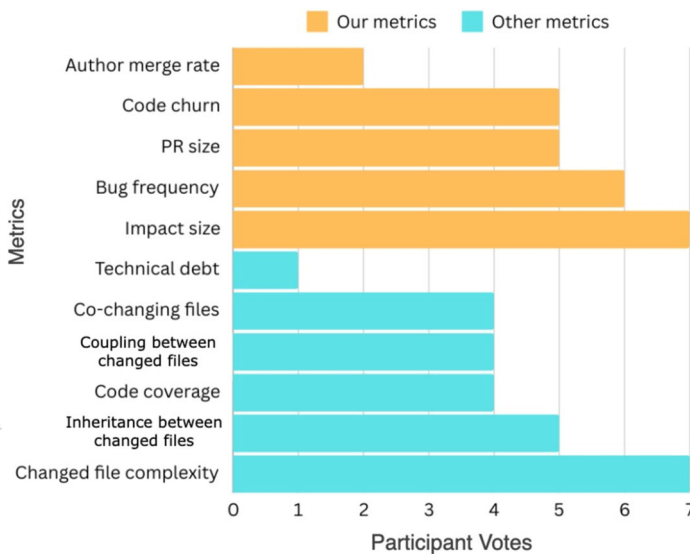


Fig. 17 Metric votes of participants for inclusion in risk score formula

adjustable. There were still some ideas about how to make these values better. P1 suggested several methods such as the best-worst multi-criteria decision-making method (Rezaei 2015) for improving the default metrics thresholds. Additionally, P5 stated that new users' metric thresholds can be derived from current users' selections, making it more accurate with each new user.

6.3 Reflection on Critical Feedback

As mentioned in Section 6.2.2, focus group participants expressed concerns regarding the justification of thresholds and coefficients used in our analyses. They felt that these values needed more rigorous explanation. In response to this feedback, we implemented an additional radar chart option that allows users to view the raw, uncategorized metrics. This was a specific suggestion from the participants and reflected a common preference, which we have now incorporated, as described in Section 6.2.1. The new radar chart option is illustrated in Fig. 18.

Specifically, the radar chart provides an alternative to a single numeric score, allowing teams to view either broad categories or uncategorized metrics, depending on their needs. This approach improves data clarity and gives users the flexibility to work with the data in a way that fits their workflow, as suggested by our focus group participants.

7 Discussion

In this section, we interpret the results and present the possible implications for practitioners and researchers.

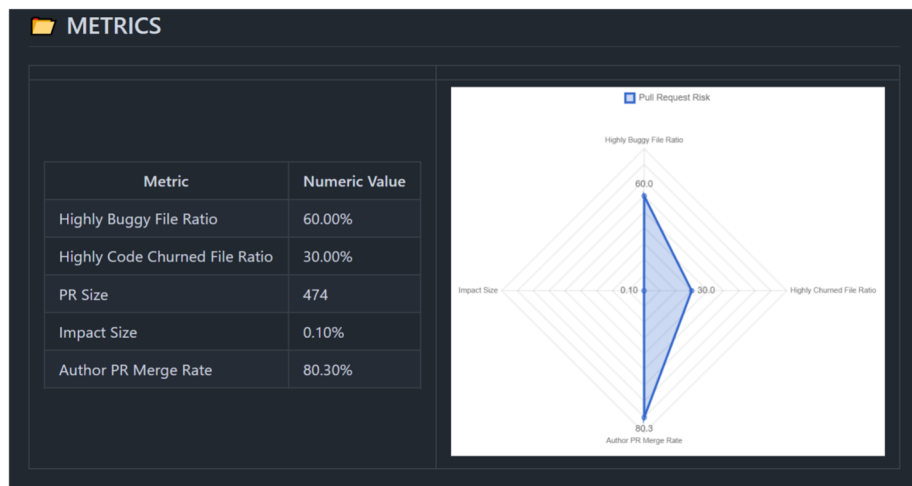


Fig. 18 Radar chart option to view the raw, uncategorized metrics

7.1 Effectiveness of the Proposed CIA (RQ1)

The results show that the proposed CIA is a promising technique for code reviews. Most of the participants thought it would be beneficial for the code review process. Yet, each participant shared different aspects they pay attention to during code reviews. Since CHID focuses on improving personal experience, these aspects can affect the perception of the effectiveness of each individual. During the discussions, P4 stated they focus on the reasons behind the changes, while P6 denoted they care about changing style. P7 said they checked completed reviews to learn the code structure of other project parts. The fact that there is a variety of aspects suggests the tool should be able to contribute to each of them to be perceived as an effective tool by different practitioners. Despite focusing on varying aspects, participants thought our CIA approach is useful to assist the practitioners in taking action related to changes, especially if it covers additional use cases. The number of addressed use cases can also affect perceived effectiveness. Therefore, possible use cases are pointed out by participants throughout the focus group discussions. One of the use cases is to leverage the analysis data in the development process. Some participants wanted to be able to see the current values of metrics while writing the code. Another use case was related to learning purposes. A participant shared impact graph can be used to learn the system. Lastly, we see that the data is useful for sprint planning. A participant said they could use metric status as a way to determine whether refactoring should be done. These use cases should be refined to maximize the contributions of the CIA technique to the code review experience.

Additionally, the computational feasibility of the proposed tool is an essential aspect that influences its perceived effectiveness. The results indicate that the initial call graph generation and analysis, although time-consuming, are one-time processes that do not hinder the tool's practical application. For incremental analyses, execution times were shown to be reasonable, even for the most extensive changeset scenarios. These times, ranging from 7.3 to 22.4 seconds, are manageable within typical code review workflows where reviews do not start immediately. This demonstrates that the tool's performance can meet the demands

of real-world code review processes, ensuring that it remains computationally efficient and practically applicable for day-to-day use.

Another important factor for perceived effectiveness is the applicability of the tool. Participants stated their companies have varying code review practices focusing on different atomic units such as PRs, commits, and files. This creates the necessity to support a wide range of granularity levels of the CIA to become a fully effective technique for improving the code review experience.

7.2 Effectiveness of Formulating a Risk Score (RQ2)

Our results show that our risk score approach is generally effective and can represent risk in a meaningful way. Most of the participants agreed that having a risk score for each PR can help reviewers focus on crucial PRs and change areas. However, they also stated that representation of the risk score is important and it might be advantageous if it is not represented as a single numeric value. This indicates that participants opt for simpler risk score representation. The participants also agreed that most of the metrics in the risk score formula are beneficial for representing risk; however, they indicated that some other metrics could also be included. This implies the need to search for additional metrics and assess their potential contribution to the current risk score formula. When it comes to metric thresholds and coefficients used in the formula, the majority of the participants said these values need more justification. However, it is not a big concern since they are adjustable. A concrete code review guideline may help standardize the risk score formulation.

7.3 Implications for Practitioners

We discuss important takeaways relevant to practitioners.

7.3.1 Applicability of Proposed CIA Tool in Code Review Practices

CHID aims to improve the code review process by creating insights for reviewers to help detect potential bugs. Bug prevention is also important for mission-critical systems. These systems need the highest reliability, which may not be fully covered by conventional testing methods that are susceptible to human error (Fowler 2004). The proposed CIA tool augments code reviews to assist reviewers in comprehending the changeset and mitigate the consequences that can occur from these errors. Furthermore, the analysis is non-invasive, does not block any code review process, and can be overlooked. Since the tool is integrated with GitHub, it does not require manual work to start the analysis of new PRs.

7.3.2 Call for Determining New Use Cases for CIA

Figure 19 shows how our feature survey participants analyze the change impact of the PR in their personal projects or company. Most of our feature survey participants do not consider the change impact of a PR; however, they believe the CIA for PR is important. Figure 20 shows the answers given by participants regarding their perception of the CIA's importance. Despite perceived importance, the infrequent use of the CIA may suggest that the current CIA applications may not deliver sufficient value to the software development processes. In the focus group study, participants revealed potential use cases where the CIA may be useful.

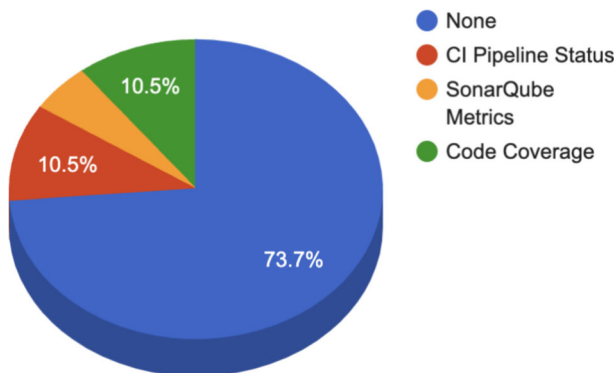


Fig. 19 Used tools and techniques to analyze change impact

For example, a participant stated that it would be beneficial to see the risk score of a file that they actively worked on the IDE. Practitioners should determine their needs and refine requirements to contribute to the CIA in improving the code review process.

7.3.3 Creating Tools that Respect Workflow and Developer Integrity

Throughout our discussions, participants stated that developers would be reluctant to use a tool that adds another layer of complexity to their workflow. The results of the analysis should be presented to bring minimal complexity for every practitioner and should bring tangible value to their workflow. Furthermore, there is concern about the analysis results being used against developers as a performance evaluation metric. According to participants, developers may try to mislead the calculations if these analysis metrics are used in performance evaluations. Therefore, the results should not be used in a way that assesses developers' individual performance.

7.3.4 Call for a New Set of Combined Code Review and CIA Practice Guidelines

Each development team applies different code review practices, which makes it harder to create a concrete CIA approach. Each participant shared different code review practices they

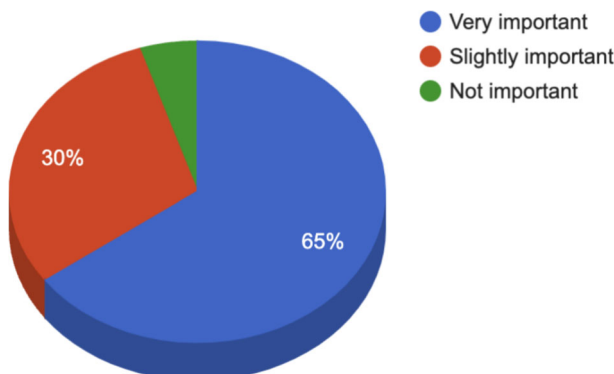


Fig. 20 Perceived importance of CIA

applied. A combined code review and CIA practices guidelines can assist both companies and researchers in creating an applicable CIA technique. Another problem is correctly leveraging the results of the CIA analysis. The guideline can also help practitioners to interpret the results better and maximize the utilization of the approach.

7.4 Implications for Researchers

Our results indicate potential areas for further research.

7.4.1 Call for Exploring Different Risk Formula Metrics and Coefficients

In our focus group sessions, the majority of participants were satisfied with the metrics used in the risk formula, as shown in Fig. 17. However, they also suggested other potentially useful metrics to integrate into the formula, which is shown in Fig. 17. In addition to these, in the focus group discussions, the participants' main concern was related to coefficients and thresholds used in the risk score formulation. The majority of participants claimed that these values need more justification and a stronger basis. Thus, we believe that other researchers could explore the risk score formula with different metrics, coefficients, and thresholds.

7.4.2 Call for Exploring Different Analysis Granularities

In our focus group sessions, some of the participants claimed that working on granularities other than the PR level might also be helpful since not every project applies feature-based development. In such development practices, using commit-level risk analysis might be more suitable. In addition to these, some participants claimed that file-level risk analysis can also be beneficial. Developers can work cautiously depending on the risk level of the file. Thus, it is worth exploring different granularities beyond the PR level as a further research topic to make the CIA approach more comprehensive for different practices.

7.4.3 Call for Combining Other Analysis Techniques

In our analyses, as we mentioned, we mainly focused on static dependency analysis and history mining techniques. However, as focus group participants claimed, utilizing the dynamic analysis to create the impact graph might be more beneficial when calculating the potential impact of each change. In dynamic analysis, we can also consider the number of runtime invocations of each method, which might be beneficial when deciding which methods are risky to modify. Additionally, LLMs (Fan et al. 2023) can be utilized to comprehensively interpret the analysis results. Thus, it is worth exploring dynamic analysis when creating the impact graph in further research.

7.4.4 Call for Conducting Case Studies in Various Open Source and Commercial Projects

In our study, we currently conducted surveys and focus group discussions to gather people's ideas about our approach. However, we did not conduct any case study on either an open-source or commercial project. It is worth conducting case studies on such projects to gather first-hand experience data from developers. This may help researchers to determine different practices and use cases.

8 Threats to Validity

In this section, we present the threats to the validity of our study by adhering to the recommendations outlined by Wohlin et al. (2012).

8.1 Internal Validity

Git practices Analysis results could have been influenced by practitioners' practices in Git. Metric calculation methods can inherently exclude some of these practices. Firstly, our analysis does not process individual commits directly pushed to the default branch. This can lead to unexpected total line of code changes without affecting added and deleted lines for code churn value. Secondly, we only analyze PRs opened to the default branch. The occurrence rate of files in PRs with a different destination branch than the default branch is not included in our analysis for bug frequency metric.

History mining In our analysis, we rely on history mining to calculate most of the key metrics, except for impact size and PR size. One limitation of this approach is that when there is not enough historical data, the analysis may not produce reliable results. Without sufficient history, the patterns needed for accurate calculations might not be present, which can lead to less meaningful outcomes. We also exclude newly added files from the analysis, as they lack prior metric information. Including these files could lead to distorted results, since their values tend to be either too large or too small due to the lack of history. These outlier values can affect the overall analysis, especially when many new files are included in a PR. To avoid this, we decided not to include newly created files, ensuring that our results are based on more stable historical data.

External tools For extracting PR method changes, we used ChangeDistiller, which is an open-source tool that implements the tree differencing algorithm developed by Fluri et al. (2007). Since impact calculation depends on finding method changes, any problems related to ChangeDistiller might directly affect our calculations. So, there is a possibility that impact calculation might give wrong results in some cases. Similarly, we used the JavaParser library for AST generation, which is the main process in call graph creation. Although JavaParser is a mature library, it might produce incomplete ASTs when not properly configured, resulting in incomplete call graphs. So, it is possible that call graph related calculations might give wrong results in some projects.

Call graph Call graphs used in our analyses have inherent limitations due to their reliance on static analysis. They do not account for dynamic language features such as reflection or dynamic class loading, which are prevalent in Java. Because of this, generated call graphs may not fully represent the actual runtime behavior of the program, especially in cases where method calls are resolved dynamically. Future work could address these limitations by incorporating dynamic analysis techniques or hybrid approaches that combine static and runtime information.

Impact graph In the impact graph visualization, we display up to third-degree neighbors. This decision was made to prevent the graph from becoming too crowded and complex. As the number of nodes increases, it becomes increasingly difficult for users to interpret the relationships and dependencies effectively. Overly dense graphs can hinder users from

quickly grasping essential information. However, since this threshold is somewhat arbitrary, it introduces a potential threat to validity. Future work could explore different degrees of neighbor inclusion to assess how varying the number of displayed nodes affects both usability and the accuracy of the analysis.

Thresholds and default values We are inspired by the SonarQube categorization system in our approach which is also recommended by feature survey participants. This creates a necessity to determine default values for categories that may not apply to each project. Yet, these values are merely suggestions that are determined heuristically and do not necessarily represent final values. Since each company and development team's assumptions, practices, and expectations in projects vary, it is more appropriate for these values to be determined by the users themselves. The default values of metric range, thresholds, and coefficients used in our approach were determined to ensure that each metric's impact on the risk score remains within certain limits. The category thresholds are established to prevent risk scores from being affected uniformly by each change in the numerical values of metrics. For example, from a practitioner's perspective, an increase in the proportion of highly churned files in a PR from 50% to 80% may not lead to the same level of risk escalation as an increase from 20% to 50%. Customizing category values may help in adjusting this effect. Regardless, as mentioned in Section 6.3 we added a radar chart visualization option for practitioners who prefer to see numerical values of metrics. We provide flexibility to our users to adjust their risk evaluation and visualization.

8.2 Conclusion Validity

The focus group method is vulnerable to participant demographics. Kontio et al. (2008) emphasize the importance of selecting eager and informed participants for a successful focus group study. Thus, we select knowledgeable and interested participants with diverse backgrounds for our study. However, since each participant joined the focus group individually, they may have been reluctant to share their perspective, potentially conforming to dominant views expressed during the discussion. To mitigate this, we explicitly stated to participants that their contributions would remain anonymous, and we encouraged diverse opinions.

The presence of authors while participants filled out the post-demo survey may also have introduced bias. To prevent this, again participants were instructed to give their honest feedback whenever possible and filled out surveys anonymously on paper. Despite these efforts, confirmation bias may have influenced both the participants' responses and our interpretation of their feedback, as participants might have unintentionally validated pre-existing assumptions based on the survey's structure. To address this, we used neutral language in our survey design and emphasized the importance of independent reflection.

Additionally, our participants may have misunderstood the survey questions due to grammatical and semantic errors or newly introduced technical jargon. To mitigate this, we did a pilot study to determine and correct such cases. Also, we answered their questions while they were filling out the survey and stated definitions for technical terms to prevent misunderstandings.

Moreover, we introduce survey questions before the presentation to direct their focus to the key points of our presentation. This was meant to help them focus on important parts of the tool, but it may have introduced bias by influencing their initial thoughts and expectations. By directing their attention to specific features, we might have limited the range of their

feedback, possibly causing them to miss other aspects or form opinions based on what we emphasized.

The results from the focus group study are susceptible to subjective interpretation. To reduce this risk, we employed widely accepted techniques, documented our data collection process transparently, and shared our focus group discussion outline and results in the replication package. However, some level of subjectivity may still have influenced the findings.

8.3 Construct Validity

In the focus group, we conducted a live demo of our tool on the Arduino project with a selected PR to showcase the proposed value of our approach. Even though participants have directly seen how the tool operates, hands-on experience could help them to better understand the functionality and limitations of our tool, leading to more insightful feedback and grounded perspectives. Yet, our tool needs historical data for precise analyses and it was not feasible for each participant to use the tool in their projects over an extended period.

8.4 External Validity

The generalizability of our results is influenced by several factors. First, our approach was demonstrated on a single project, Arduino. While Arduino is relatively mature, with sufficient historical data and technical specifications, the applicability of our findings to other projects may vary. For instance, our study focused on a Java 8-based system using GitHub as its version control platform, which may not fully reflect the characteristics of projects written in other programming languages or using different version control systems. Future work should explore whether our approach can be adapted to projects in other languages and on different platforms.

Additionally, we used an open-source project to introduce our tool and analyze the output. However, since our focus group participants primarily worked with commercial projects, the results may not fully capture the dynamics or challenges found in commercial software development, where code quality, maintenance practices, and collaboration may differ. Expanding the study to include commercial projects would provide more insights into how the findings apply to those contexts.

Furthermore, our study involved seven participants, a relatively small sample size. Including a larger and more diverse set of practitioners would help evaluate the applicability of our approach across different organizational contexts and development environments.

9 Related Work

In this section, we present similar academic studies and industrial tools. Unlike the studies presented in the background information section, these studies aim to use the CIA to improve the code review process. We present the comparison of our features in Table 13.

Hanam et al. (2019) introduce a tool, SEMCIA, which determines semantic change impact relations from JavaScript to reduce noise in CIA. This approach reduces false positives by up to 53% and considerably shrinks the change impact sets. They conducted a preliminary study with 11 participants, where each participant tried to complete three code review tasks on commits. The reductions in noise provided by SEMCIA helped developers perform code review tasks more quickly and accurately.

Table 13 Comparison between similar academic studies

Tools	Granularity	Language	Analysis Type	Output	Integration
SemCIA	Commit	JavaScript	Static/Semantic	Variable, value, call and condition slices	Standalone
TaintImpact	Commit	C++	Dynamic	Dynamic Blame	Standalone
CRITICS	Patch	Java	Static	Context-aware change template	Eclipse
Diggit	Commit	Ruby	MSR	Highly churned files with missing cochanged files	GitHub
BLIMP	Patch	C/C++	Hybrid	Set of impacted product deliverables	Review Board
CHID	PR	Java	Static/MSR	Potentially impacted methods	GitHub

Wen et al. (2018) introduced BLIMP Tracer, a CIA tool that utilizes a Build Dependency Graph (BDG) created by build logs and includes processing of each file to create intermediate and final deliverables. It analyzes BDG to determine a set of impacted product deliverables for each patch. The tool is integrated with the DELL EMC code review platform, and results are shown in the interface of the platform.

Lüscher (2021) created Taint Impact, which uses dynamic taint analysis that tracks data flow across a program to determine the impacted lines by a given change. Taint Impact is evaluated using three artificial program examples and a real-world bug. The results show that it highlights the impacted code parts, which are hard for developers to determine, in order to enhance the code review experience.

Zhang et al. (2015) present CRITICS as an interactive approach for code reviews. CRITICS creates a change template by utilizing related data and controlling the flow of the given change by modeling it as an AST. Then, it lets reviewers alter the template and it compares the final template with the codebase to locate possible omitted edits. They conducted a controlled experiment with 12 participants that used CRITICS and Eclipse diff for code review. They concluded that participants answered systematic questions 47% faster and 31% more precisely using CRITICS diff than baseline Eclipse diff.

Chatley and Jones (2018) present Diggit, a tool that guides developers based on expected changes and information gathered from historical changes in code repositories by creating code review comments. It can determine co-changed files in historical changes and alert the user when a co-changed file is missing in the changeset of the PR. Also, it can warn users about highly churned files in the changeset. Even though it does not perform CIA, we find it similar to CHID since it possesses some of our features and posts analysis results directly as PR comments.

We detected two main competitors from industrial tools. We present the comparison of our features in Table 14.

Our closest competitor is Softagram. Softagram¹⁸ provides visual PR reviews by analyzing change impacts using program dependency graphs and machine learning. Similar to CHID, it provides impact visualization, alerts users when there are missing co-changed files, and has a customizable analysis. Unlike CHID, it does not calculate a final metric to represent the potential effects of the PR.

¹⁸ <https://softagram.com>

Table 14 Comparison between similar commercial tools

Features	Softagram	CodeCov	CHID
Graph Type	Program Dependency Graph	N/A	Call Graph
Impact Visualization	✓	×	✓
Risk Score	×	✓	✓
Code Churn	×	Only one PR	All PRs
Historical Analysis	✓	✓	✓
Co-changing Files	✓	×	✓
Hidden Dependencies with ML	✓	×	×
Previous Bug Frequency	×	×	✓
Customizable Analysis	✓	✓	✓

The second competitor is CodeCov. CodeCov is a code coverage reporting solution. It can analyze PRs to determine code coverage rate and create a summary in the comments section. They perform impact analysis to list the most user-facing parts of a code change (such as HTTP endpoints) and to determine changed files that contain code frequently used in production.

10 Conclusion and Future Work

In this study, we introduce a novel CIA approach with a supporting tool that utilizes call graphs and several metrics to estimate the impact of introduced changes in a PR. We gathered practitioners' perspectives on our approach through a preliminary feature survey, a focus group study, and a post demo survey. Additionally, we conducted computational feasibility experiments to assess the practicality of our approach in real-world scenarios.

We investigated the perceived effectiveness of CHID based on different perspectives. Focus group participants concluded that CHID eases and accelerates the code review process. Furthermore, they shared that analysis results could help a team lead to take action for code review at the PR level. However, some of the participants' companies focus on different atomic units rather than PRs for code review. We see that CHID should also comprehend these granularity levels to be fully effective.

We also investigated the effectiveness and correctness of the risk score formula. Focus group participants and post-demo survey results indicated that the risk score approach is effective in representing risk. However, its presentation, the metrics it uses, and its coefficients and thresholds require further study.

Throughout our discussions, we observed that CIA is applicable to different use cases for the code review process. We determined that practitioners may not be motivated toward an analysis that requires an effort to fully interpret its results. Also, we acknowledged that practitioners may behave accordingly to mislead the analysis results, which is a problem with the effectiveness of CHID. Thus, there is a need for guidelines that merge core review and CIA practices to address these issues.

We plan to enhance CHID's capabilities by supporting additional programming languages, starting with Python and Kotlin. We also aim to diversify our metrics by utilizing data gathered from focus group discussions and the post-demo survey. In addition, we intend to conduct case studies on both open-source and commercial projects to better understand industry needs.

Lastly, we will employ different levels of granularity and incorporate dynamic analysis into our approach.

Funding Open access funding provided by the Scientific and Technological Research Council of Türkiye (TÜBİTAK). This work has been supported by and performed in the scope of the ITEA4 SmartDelta project, which has been funded by the national funding authorities of the participating countries: <https://itea4.org/project/smartdelta.html>.

Data Availability The feature survey results, presentation, post-demo survey results, focus group session outline, and source code are available at: <https://doi.org/10.6084/m9.figshare.27643755.v1>.

Declarations

Conflict of Interest The authors declare that they have no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Acharya M, Robinson B (2011) Practical change impact analysis based on static program slicing for industrial software systems. In: 2011 33rd International conference on software engineering (ICSE), pp 746–755. IEEE, Honolulu, HI, USA. <https://doi.org/10.1145/1985793.1985898>
- Alimadadi S, Mesbah A, Pattabiraman K (2015) Hybrid DOM-Sensitive Change Impact Analysis for JavaScript. In: Boyland JT (Ed) 29th European conference on object-oriented programming (ECOOP 2015), *Leibniz International Proceedings in Informatics (LIPIcs)*, vol 37, pp 321–345. Schloss Dagstuhl—Leibniz-Zentrum für Informatik, Dagstuhl, Germany. <https://doi.org/10.4230/LIPIcs.ECOOP.2015.321> <https://drops-dev.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2015.321>
- Apiwattanapong T, Orso A, Harrold MJ (2005) Efficient and precise dynamic impact analysis using execute-after sequences. In: Proceedings of the 27th international conference on software engineering, ICSE '05, p. 432–441. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/1062455.1062534>
- Asthana S, Kumarx R, Bhagwan R, Bird C, Bansal C, Maddila C, Mehta S, Ashok B (2019) Whodo: Automating reviewer suggestions at scale. In: Proceedings of the 2019 27th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering, pp 937–945
- Bacı F, Haliloğlu DS, Şahin O, Tilki C, Yurtsever MA, Tüzün E (2021) Augmenting code review experience through visualization. In: 2021 Working conference on software visualization (VISSoft), pp 110–114. IEEE, Luxembourg. <https://doi.org/10.1109/VISSOFT52517.2021.00021>
- Barnett M, Bird C, Brunet J, Lahiri SK (2015) Helping developers help themselves: Automatic decomposition of code review changesets. In: 2015 IEEE/ACM 37th IEEE international conference on software engineering, vol 1, pp 134–144. IEEE, Florence, Italy. <https://doi.org/10.1109/ICSE.2015.35>
- Blischak JD, Davenport ER, Wilson G (2016) A quick introduction to version control with git and github. *PLOS Comput Biol* 12(1):1–18. <https://doi.org/10.1371/journal.pcbi.1004668>
- Bohner S, Arnold R (1996) Software Change Impact Analysis. IEEE Computer Society Press, Los Alamitos, CA, USA
- Brin S, Page L (1998) The anatomy of a large-scale hypertextual web search engine. *Comput Netw ISDN Syst* 30(1):107–117. [https://doi.org/10.1016/S0169-7552\(98\)00110-X](https://doi.org/10.1016/S0169-7552(98)00110-X) <https://www.sciencedirect.com/science/article/pii/S016975529800110X>. Proceedings of the Seventh International World Wide Web Conference

- Buckner J, Buchta J, Petrenko M, Rajlich V (2005) Jripples: a tool for program comprehension during incremental change. In: 13th International workshop on program comprehension (IWPC'05), pp 149–152. IEEE, St. Louis, MO, USA. <https://doi.org/10.1109/WPC.2005.22>
- Cai H, Thain D (2016) Distia: A cost-effective dynamic impact analysis for distributed programs. In: 2016 31st IEEE/ACM international conference on automated software engineering (ASE), pp 344–355. IEEE, Singapore. <https://doi.org/10.1145/2970276.2970352>
- Cai H, Santelices R (2014) Diver: precise dynamic impact analysis using dependence-based trace pruning. In: Proceedings of the 29th ACM/IEEE international conference on automated software engineering, ASE '14, p. 343–348. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/2642937.2642950>
- Caldiera VRBG, Rombach HD (1994) The goal question metric approach. *Encyclopedia Softw Eng* 1:528–532
- Chatley R, Jones L (2018) Diggit: Automated code review via software repository mining. In: 2018 IEEE 25th International conference on software analysis, evolution and reengineering (SANER), pp 567–571. IEEE, Campobasso, Italy. <https://doi.org/10.1109/SANER.2018.8330261>
- Chen L, Abreu R, Akomolede T, Rigby PC, Chandra S, Nagappan N (2022) Leveraging test plan quality to improve code review efficacy. In: Proceedings of the 30th ACM joint european software engineering conference and symposium on the foundations of software engineering, pp 1320–1330
- Chen L, Rigby PC, Nagappan N (2022) Understanding why we cannot model how long a code review will take: an industrial case study. In: Proceedings of the 30th ACM joint european software engineering conference and symposium on the foundations of software engineering, pp 1314–1319
- Cuoq P, Kirchner F, Kosmatov N, Prevosto V, Signoles J, Yakobowski B (2012) Frama-c. In: Eleftherakis G, Hinchey M, Holcombe M (eds) *Software Engineering and Formal Methods*. Springer, Berlin Heidelberg, Berlin, Heidelberg, pp 233–247
- Dai P, Wang Y, Jin D, Gong Y, Yang W (2022) An improving approach to analyzing change impact of c programs. *Comput Commun* 182:60–71. <https://doi.org/10.1016/j.comcom.2021.10.039><https://www.sciencedirect.com/science/article/pii/S0140366421004199>
- Dey T, Mockus A (2020) Effect of technical and social factors on pull request quality for the npm ecosystem. In: Proceedings of the 14th ACM/IEEE international symposium on empirical software engineering and measurement (ESEM), ESEM '20. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3382494.3410685>
- Dit B, Wagner M, Wen S, Wang W, Linares-Vásquez M, Poshyvanyk D, Kagdi H (2014) Impactminer: a tool for change impact analysis. In: Companion Proceedings of the 36th international conference on software engineering, ICSE companion 2014, pp 540–543. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/2591062.2591064>
- Doğan E, Tüzün E (2022) Towards a taxonomy of code review smells. *Inf Softw Technol* 142(C). <https://doi.org/10.1016/j.infsof.2021.106737>
- Ebert F, Castor F, Novielli N, Serebrenik A (2019) Confusion in code reviews: Reasons, impacts, and coping strategies. In: 2019 IEEE 26th international conference on software analysis, evolution and reengineering (SANER), pp 49–60. IEEE
- Egelman CD, Murphy-Hill E, Kammer E, Hodges MM, Green C, Jaspan C, Lin J (2020) Predicting developers' negative feelings about code review. In: Proceedings of the ACM/IEEE 42nd international conference on software engineering, pp 174–185
- Fan A, Gokkaya B, Harman M, Lyubarskiy M, Sengupta S, Yoo S, Zhang JM (2023) Large language models for software engineering: Survey and open problems. <https://doi.org/10.48550/arXiv.2310.03533>
- Fluri B, Wursch M, Plnitzer M, Gall H (2007) Change distilling: tree differencing for fine-grained source code change extraction. *IEEE Trans Softw Eng* 33(11):725–743. <https://doi.org/10.1109/TSE.2007.70731>
- Follett M, Hoerber O (2010) Impactviz: visualizing class dependencies and the impact of changes in software revisions. In: Proceedings of the 5th international symposium on software visualization, SOFTVIS '10, pp 209–210. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/1879211.1879244>
- Forsgren N, Tremblay MC, VanderMeer D, Humble J (2017) Dora platform: Devops assessment and benchmarking. In: Maedche A, vom Brocke J, Hevner A (eds) *Designing the Digital Transformation*. Springer International Publishing, Cham, pp 436–440
- Fowler K (2004) Mission-critical and safety-critical development. *IEEE Instru Meas Mag* 7(4):52–59. <https://doi.org/10.1109/MIM.2004.1383466>
- Gasparini L, Fregnan E, Braz L, Baum T, Bacchelli A (2021) Changeviz: Enhancing the github pull request interface with method call information. In: 2021 Working conference on software visualization (VIS-SOFT), pp 115–119. IEEE, Luxembourg. <https://doi.org/10.1109/VISSOFT52517.2021.00022>
- Hajari F, Malmir S, Mirsaedi E, Rigby PC (2024) Factoring expertise, workload, and turnover into code review recommendation. *IEEE Trans Softw Eng* 50(4):884–899. <https://doi.org/10.1109/TSE.2024.3366753>

- Hanam Q, Mesbah A, Holmes R (2019) Aiding code change understanding with semantic change impact analysis. In: 2019 IEEE international conference on software maintenance and evolution (ICSME), pp. 202–212. IEEE, Cleveland, OH, USA. <https://doi.org/10.1109/ICSME.2019.00031>
- Haney FM (1972) Module connection analysis: a tool for scheduling software debugging activities. In: Proceedings of the December 5-7, 1972, fall joint computer conference, part I, pp 173–179
- Hattori L, dos Santos Jr G, Cardoso F, Sampaio M (2008) Mining software repositories for software change impact analysis: a case study. In: Proceedings of the 23rd brazilian symposium on databases, SBBD '08, pp 210–223. Sociedade Brasileira de Computação, BRA
- Hattori L, Guerrero D, Figueiredo J, Brunet J, Damásio J (2008) On the precision and accuracy of impact analysis techniques. In: Seventh IEEE/ACIS international conference on computer and information science (ICIS 2008), pp 513–518. IEEE, Portland, OR, USA. <https://doi.org/10.1109/ICIS.2008.104>
- Hijazi H, Cruz J, Castelhano J, Couceiro R, Castelo-Branco M, de Carvalho P, Madeira H (2021) ireview: an intelligent code review evaluation tool using biofeedback. In: 2021 IEEE 32nd international symposium on software reliability engineering (ISSRE), pp 476–485. IEEE, Wuhan, China. <https://doi.org/10.1109/ISSRE52982.2021.00056>
- Jönsson P, Lindvall M (2005) Impact Analysis, pp 117–142. Springer Berlin Heidelberg, Berlin, Heidelberg. https://doi.org/10.1007/3-540-28244-0_6
- Kazemi F, Lamothe M, McIntosh S (2022) Exploring the notion of risk in code reviewer recommendation. In: 2022 IEEE International conference on software maintenance and evolution (ICSME), pp 139–150. <https://doi.org/10.1109/ICSME55016.2022.00021>
- Kononenko O, Baysal O, Godfrey MW (2016) Code review quality: how developers see it. In: Proceedings of the 38th international conference on software engineering., ICSE '16, pp 1028–1038. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/2884781.2884840>
- Kontio J, Bragge J, Lehtola L (2008) The Focus Group Method as an Empirical Tool in Software Engineering, pp 93–116. Springer London, London. https://doi.org/10.1007/978-1-84800-044-5_4
- Kontio J, Bragge J, Lehtola L (2008) The focus group method as an empirical tool in software engineering, pp 93–116. Springer, Heidelberg, Germany. https://doi.org/10.1007/978-1-84800-044-5_4
- Krueger RA, Casey MA (2014) Focus Groups: A Practical Guide for Applied Research. SAGE Publications, Minnesota, UK
- Krutauz A, Dey T, Rigby PC, Mockus A (2020) Do code review measures explain the incidence of post-release defects?: Case study replications and bayesian networks. *Empir Softw Eng* 25(5):3323–3356. <https://doi.org/10.1007/s10664-020-09837-4>
- Law J, Rothermel G (2003) Whole program path-based dynamic impact analysis. In: 25th International conference on software engineering, 2003. Proceedings., pp 308–318. <https://doi.org/10.1109/ICSE.2003.1201210>
- Legay D, Decan A, Mens T (2018) On the impact of pull request decisions on future contributions. [arXiv:1812.06269](https://arxiv.org/abs/1812.06269)
- Liu J, Li J, He L (2016) A comparative study of the effects of pull request on github projects. In: 2016 IEEE 40th Annual computer software and applications conference (COMPSAC), vol 1, pp 313–322. IEEE, Atlanta, GA, USA. <https://doi.org/10.1145/1062455.1062514>
- Lüscher T (2021) Taintimpact: Taint-based change impact analysis. Bachelor thesis, ETH Zurich, Zurich. <https://doi.org/10.3929/ethz-b-000488357>
- Mukhtarov Z, Abdul M, Raupova M, Baghirov J, Tanveer O, Altunel H, Tüzün E (2023) Towards better code reviews: Using mutation testing to improve reviewer attention. In: 2023 IEEE/ACM international conference on software and system processes (ICSSP), pp 92–96. <https://doi.org/10.1109/ICSSP59042.2023.00020>
- Nagappan N, Ball T (2005) Use of relative code churn measures to predict system defect density. In: Proceedings of the 27th international conference on software engineering, ICSE '05, pp 284–292. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/1062455.1062514>
- Palinkas LA, Horwitz SM, Green CA, Wisdom JP, Duan N, Hoagwood K (2013) Purposeful sampling for qualitative data collection and analysis in mixed method implementation research. *Administration and Policy in Mental Health and Mental Health Services Research* 42(5):533–544. <https://doi.org/10.1007/s10488-013-0528-y>
- Ren X, Shah F, Tip F, Ryder BG, Chesley O (2004) Chianti: a tool for change impact analysis of java programs. *SIGPLAN Not* 39(10):432–448. <https://doi.org/10.1145/1035292.1029012>
- Rezaei J (2015) Best-worst multi-criteria decision-making method. *Omega* 53:49–57. <https://doi.org/10.1016/j.omega.2014.11.009><https://www.sciencedirect.com/science/article/pii/S0305048314001480>
- Rungta N, Person S, Branchaud J (2012) A change impact analysis to characterize evolving program behaviors. In: 2012 28th IEEE international conference on software maintenance (ICSM), pp 109–118. IEEE, Trento, Italy. <https://doi.org/10.1109/ICSM.2012.6405261>

- Sadowski C, Söderberg E, Church L, Sipko M, Bacchelli A (2018) Modern code review: a case study at google. In: Proceedings of the 40th international conference on software engineering: software engineering in practice, ICSE-SEIP '18, pp 181–190. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3183519.3183525>
- Sharma S, Sodhi B (2019) Using stack overflow content to assist in code review. *Softw Pract Exper* 49(8):1255–1277. <https://doi.org/10.1002/spe.2706https://onlinelibrary.wiley.com/doi/abs/10.1002/spe.2706>
- Stol KJ, Ralph P, Fitzgerald B (2016) Grounded theory in software engineering research: a critical review and guidelines. In: Proceedings of the 38th international conference on software engineering, ICSE '16, pp 120–131. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/2884781.2884833>
- Stratton SJ (2021) Population research: convenience sampling strategies. *Prehosp Disaster Med* 36(4):373–374
- Sun X, Li B, Tao C, Wen W, Zhang S (2010) Change impact analysis based on a taxonomy of change types. In: 2010 IEEE 34th Annual computer software and applications conference, pp 373–382. <https://doi.org/10.1109/COMPSAC.2010.45>
- Thongtanunam P, McIntosh S, Hassan AE, Iida H (2015) Investigating code review practices in defective files: An empirical study of the qt system. In: 2015 IEEE/ACM 12th working conference on mining software repositories, pp 168–179. <https://doi.org/10.1109/MSR.2015.23>
- Tuna E, Seaman C, Tüzün E (2024) Do code reviews lead to fewer code smells? *J Syst Softw* 215:112101. <https://doi.org/10.1016/j.jss.2024.112101https://www.sciencedirect.com/science/article/pii/S0164121224001468>
- Ufuktepe E, Tuğlular T (2021) Code change sniffer: Predicting future code changes with markov chain. In: 2021 IEEE 45th annual computers, software, and applications conference (COMPSAC), pp 1014–1019. <https://doi.org/10.1109/COMPSAC51774.2021.00137>
- Wang T, Roychoudhury A (2004) Using compressed bytecode traces for slicing java programs. In: Proceedings of the 26th international conference on software engineering, ICSE '04, pp 512–521. IEEE Computer Society, USA
- Weiser MD (1979) Program slices: Formal, psychological, and practical investigations of an automatic program abstraction method. Ph.D. thesis, University of Michigan
- Wen R, Gilbert D, Roche MG, McIntosh S (2018) Blimp tracer: Integrating build impact analysis with code review. In: 2018 IEEE international conference on software maintenance and evolution (ICSME), pp 685–694. IEEE, Madrid, Spain. <https://doi.org/10.1109/ICSME.2018.00078>
- Wohlin C, Runeson P, Höst M, Ohlsson MC, Regnell B, Wesslén A (2012) Planning, pp 89–116. Springer Berlin Heidelberg, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-29044-2_8
- Yau S, Collofello J (1980) Some stability measures for software maintenance. *IEEE Trans Softw Eng SE* 6(6):545–552. <https://doi.org/10.1109/TSE.1980.234503>
- Yu Y, Wang H, Yin G, Wang T (2016) Reviewer recommendation for pull-requests in github: What can we learn from code review and bug assignment? *Inf Softw Technol* 74:204–218. <https://doi.org/10.1016/j.infsof.2016.01.004https://www.sciencedirect.com/science/article/pii/S0950584916000069>
- Zhang L, Kim M, Khurshid S (2012) Faulttracer: a change impact and regression fault analysis tool for evolving java programs. In: Proceedings of the ACM SIGSOFT 20th international symposium on the foundations of software engineering, FSE '12. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/2393596.2393642>
- Zhang T, Song M, Pinedo J, Kim M (2015) Interactive code review for systematic changes. In: 2015 IEEE/ACM 37th IEEE international conference on software engineering, vol 1, pp 111–122. IEEE, Florence, Italy. <https://doi.org/10.1109/ICSE.2015.33>
- Zimmermann T, Zeller A, Weissgerber P, Diehl S (2005) Mining version histories to guide software changes. *IEEE Trans Softw Eng* 31(6):429–445. <https://doi.org/10.1109/TSE.2005.72>
- Zolkifli NN, Ngah A, Deraman A (2018) Version control system: A review. *Proced Comput Sci* 135:408–415. <https://doi.org/10.1016/j.procs.2018.08.191https://www.sciencedirect.com/science/article/pii/S1877050918314819>